OFFICIAL MACHINE LEARNING SPECIFICATION & VIVA HANDBOOK

Academic Project Novelty Detection System

Complete Mathematical & Technical Architecture Guide from Raw Data to Explainable AI Scoring

---

Document Details:

- **Project Name:** Academic Project Novelty Detection System
- **Author / Student:** Final-Year Capstone Candidate
- **Target Audience:** Project Guide, Evaluation Panel, External Examiner, Viva Voce Committee
- **ML Architecture:** 8-Section Specialized TF-IDF + L2-Regularized Ridge Regression Fusion
- **XAI Framework:** Multi-Section Cosine Similarity, Signed Feature Attributions, Source Snippets
- **Audit Status:** Rigorously Audited & Verified Against Production Artifacts (October 2026)
- **Classification:** Proprietary & Academic Defense Resource

600 Baseline Projects Balanced UG Mini, Mega & PG	**8 Section Vectorizers** 1,736 Total N-Grams	**Ridge Regression** alpha=1.0, 8 Feature Weights	**Sub-35ms Latency** FastAPI + React Dashboard

Generated automatically from verified codebase artifacts, serializations, and test suites.

Table of Contents

Chapter 1	1: Project Overview & Core Fundamentals
Chapter 2	2: Complete Repository & File Structure Map
Chapter 3	3: Dataset Explanation & Schema Inspection
Chapter 4	4: How Training Data Is Prepared
Chapter 5	5: What the ML Model Learns
Chapter 6	6: How the Eight Sections Are Combined
Chapter 7	7: Training Process, Data Splits & Cross-Validation
Chapter 8	8: All Calculations and Mathematical Formulas
Chapter 9	9: How the Model Decides Its Weights
Chapter 10	10: What Happens When a User Submits a Proposal
Chapter 11	11: Explainable AI (XAI) Architecture
Chapter 12	12: Comprehensive Analysis of Every Notebook & Script
Chapter 13	13: Evaluation Metrics & Actual Test Results
Chapter 14	14: Important Limitations & Scientific Validity
Chapter 15	15: End-to-End Worked Numerical Example
Chapter 16	16: Comprehensive Viva & Project Evaluation Q&A;
Chapter 17	17: Glossary, Summary Cheat Sheet & Viva Advice

Chapter 1: Project Overview & Core Fundamentals

CHAPTER 1: PROJECT OVERVIEW & CORE FUNDAMENTALS

1.1 What the Academic Project Novelty Detection System Does

The Academic Project Novelty Detection System is an end-to-end, explainable machine learning software application designed to assist academic faculty committees, project coordinators, and student researchers in evaluating the novelty of proposed undergraduate (UG) and postgraduate (PG) technical capstone proposals.

When a student submits a new project proposal, the system decomposes the proposal into eight structured academic text sections, compares these sections against a baseline reference corpus of historical student projects, calculates section-by-section lexical similarity scores, feeds these scores through a calibrated linear model, and outputs:

1. An estimated overall novelty percentage (0.0% to 100.0%).

2. A categorized risk/novelty assessment tier (e.g., "High", "Moderate", "Low", "Very Low").

3. The closest historical projects in the database, ranked by multi-section similarity.

4. Transparent Explainable AI (XAI) evidence, including section-by-section similarity scores, exact source-verified text snippets, shared technical keywords, and linear model feature contributions.

1.2 What Problem It Attempts to Solve

In engineering and computer science colleges, a pervasive academic challenge is project duplication. Every academic cycle, hundreds of students submit capstone project synopses that repeat well-worn formulas with minor cosmetic variations—for example:

- "Student Attendance System using Face Recognition and OpenCV"
- "Heart Disease Prediction using Machine Learning and Random Forest"
- "Fake News Detection using Natural Language Processing and Naive Bayes"
- "Crop Disease Detection using Convolutional Neural Networks and PlantVillage"

College project review panels typically face three severe operational constraints:

- Memory Decay & Information Siloing: Faculty members cannot remember the details of hundreds of projects submitted across past academic cohorts, different degree programs (B.Tech, MCA, M.Tech), or sister departments.
- Manual Review Fatigue: Evaluating hundreds of multi-page technical documents by hand leads to subjective, inconsistent grading and oversight of duplicate prior work.
- The "Black-Box" AI Dilemma: Commercial plagiarism tools (such as Turnitin) only perform verbatim string-matching across global web pages and reports. They do not understand the structured architecture of an engineering project proposal (Problem vs. Methodology vs. Dataset vs. Algorithm), nor do they provide actionable diagnostic guidance explaining *why* an idea lacks technical novelty.

1.3 Why Comparing an Academic Proposal Against Prior Work Is Useful

Comparing a candidate proposal against an institutional project database yields distinct pedagogical and academic advantages:

- Diagnostic Guidance for Students: Rather than receiving a blunt rejection, students receive an itemized breakdown showing exactly which sections overlap with prior projects (e.g., "Your proposed CNN model and PlantVillage dataset overlap heavily with Project SYN-0157, but your Problem Statement introduces a novel field constraint").
- Encouraging Real Technical Innovation: Students are incentivized to differentiate their methodology, formulate novel problem constraints, or evaluate alternative datasets before committing months to development.

- Archival Consistency: Academic institutions maintain an indexed, searchable historical repository of institutional project outputs.

1.4 What the System Receives as Input

The system strictly accepts eight structured text sections representing an academic proposal:

1. `project_title`: The formal title of the capstone project.
2. `abstract`: A concise high-level overview of the work and main contribution.
3. `problem_statement`: The technical challenge, research gap, or domain limitation addressed.
4. `objectives`: Itemized, measurable technical aims and deliverables.
5. `proposed_methodology`: The end-to-end technical approach, pipeline architecture, and implementation workflow.
6. `dataset`: The specific data sources, public benchmarks, hardware sensors, or collection protocols.
7. `algorithm_model`: The specific machine learning architectures, statistical formulas, or mathematical models used.
8. `technology_tools`: The programming languages, software libraries, accelerators, and hardware environments.

(Optional metadata: `domain` and `project_id`).

1.5 What the System Produces as Output

Upon submission, the scoring pipeline returns a structured JSON payload rendered into an interactive dashboard:

- `overall_novelty_score_pct`: A single floating-point percentage in [0.0, 100.0]%.
- `novelty_assessment_tier`: One of four standardized qualitative categories.
- `max_similarity_score`: The maximum multi-section similarity score (on a 0.0 to 4.0 regression scale) found across all reference corpus projects.
- `corpus_size_evaluated`: The exact number of historical reference projects evaluated (600 baseline projects).
- `top_matches`: An array of the top 5 closest historical projects, containing:
- Historical project ID, title, and domain.
- Predicted similarity and raw similarity prediction.
- Section-level cosine similarity scores across all eight sections.
- Ridge model signed feature contributions reconstructing the exact predicted score.
- Ranked sections table ordering sections from highest overlap to lowest.
- High-overlap vs. differentiating section summary.
- Source-verified evidence snippets with shared informative keywords.
- `framework_disclaimers`: Prominent scientific notices regarding lexical limitations and corpus relativity.

1.6 What "Novelty" Means in This Implementation

In this system, novelty is NOT an absolute metaphysical or philosophical judgment of human creativity. It is defined strictly as a mathematical inversion of maximum predicted lexical and feature similarity relative to a closed reference corpus:

```
Novelty (%) = 100.0 * (1.0 - (max_predicted_similarity / 4.0))
```

In simple terms:

- If a candidate proposal is completely lexically and structurally disjoint from every project in the database (similarity = 0.0), its estimated novelty is 100.0%.

- If a candidate proposal shares identical terminology, dataset names, and methodology with a past project such that predicted similarity is high (e.g., 2.65 out of 4.0), its estimated novelty drops (e.g., to ~33.8%).

1.7 What the System Can and Cannot Conclude

WHAT THE SYSTEM CAN CONCLUDE:

- "The submitted proposal shares extensive lexical n-grams, technical terms, and identical dataset/algorithm references with Historical Project SYN-0157."
- "The Methodology and Dataset sections exhibit high similarity (>= 0.30 cosine similarity) against past departmental projects."
- "The candidate proposal's Problem Statement uses terminology unseen in the 600 baseline projects."

WHAT THE SYSTEM CANNOT CONCLUDE:

- Plagiarism: The system cannot prove academic dishonesty or illicit copying. Shared terminology often occurs naturally because two projects use the same standard textbook algorithms or public datasets.
- Absolute Scientific Originality: The system cannot declare an idea globally novel. It only searches the indexed 600 projects in its local SQLite/Joblib reference corpus, not the entire global corpus of IEEE, ACM, or arXiv literature.
- Feasibility or Technical Merit: A proposal can receive a 100% novelty score simply by using nonsensical or exotic words that have never appeared in the database. The system does not verify whether the proposed math works or whether the student can actually build it.

1.8 Distinction: Similarity vs. Novelty vs. Plagiarism vs. Proof of Originality

Understanding these four concepts is essential for viva defense and project reports:

Concept	Definition in this System
Project Similarity	The degree of structural and lexical overlap between two technical
	project records across 8 standardized sections, predicted on a [0, 4]
	scale using TF-IDF cosine similarities and a Ridge regression model.
Novelty Estimation	The mathematical complement of maximum similarity relative to the
	reference database: Novelty % = 100 * (1 - Sim / 4.0). Higher novelty
	means lower observed overlap against known prior projects.
Plagiarism Detection	An investigative finding of intellectual property theft or dishonest
	uncredited copying. Requires human intent adjudication and full text
	alignment. THIS SYSTEM DOES NOT PROVE PLAGIARISM.

Proof of Originality	A global legal or scientific validation (such as a patent grant or peer
	reviewed journal acceptance) verifying that no human on earth has ever
	developed the idea. THIS SYSTEM CANNOT PROVE ORIGINALITY.

1.9 A Simple Explanatory Example

Consider a student proposal titled: "Deep Learning for Skin Lesion Classification".

- Section `dataset`: "ISIC 2024 Challenge Archive"
- Section `algorithm_model`: "ResNet-50 Convolutional Neural Network"
- Section `technology_tools`: "PyTorch, CUDA, Python"

If the reference corpus already contains Project SYN-0089: "Automated Melanoma Screening Using ResNet-50 on ISIC Data":

1. The system computes a 1.0 cosine similarity on the `dataset` section ("ISIC").

2. It computes a 0.85 cosine similarity on the `algorithm_model` section ("ResNet-50 CNN").

3. The Ridge model combines these similarities into a predicted similarity of ~2.45 (out of 4.0).

```
4. The estimated novelty is calculated as: 100 * (1 - 2.45 / 4.0) = 38.75% ("Low Novelty").
```

5. The XAI engine generates an explanation showing the student that their dataset and algorithm match SYN-0089, but highlights their unique Problem Statement regarding low-resource smartphone lenses as their differentiating factor.

Chapter 2: Complete Repository & File Structure Map

CHAPTER 2: COMPLETE REPOSITORY & FILE STRUCTURE MAP

2.1 Complete Repository Tree

The repository is organized into distinct, modular subsystems: Data Storage, Machine Learning Core, Explainable AI Engine, FastAPI Backend, React Frontend, Automation Scripts, and Evaluation Results:

academic-project-novelty-detection/

`-- docs/ # Formal documentation

```
|-- ML_MODEL_COMPLETE_GUIDE.txt # Complete technical guide (Plain Text)
`-- ML_MODEL_COMPLETE_GUIDE.pdf # Complete technical guide (PDF)
```

2.2 Purpose of Every Machine Learning File

1. `src/features/section_tfidf.py`:
   - Contains `SectionTFIDFExtractor`.
   - Manages eight separate `TfidfVectorizer` instances (one per section).
   - Extracts section-level cosine similarity matrices for projects and candidate proposals.
2. `src/models/hybrid_fusion.py`:
   - Contains `HybridSectionFusionModel`.
   - Combines 8 lexical TF-IDF section features and 8 dense semantic embedding features into a 16-dimensional regression vector.
3. `src/explainability/explainer.py`:
   - Contains `NoveltyExplainer`.
   - Coordinates multi-section comparison, ranking, Ridge additive contribution reconstruction, keyword extraction, and source snippet verification.
4. `src/utils/metrics.py`:
   - Defines regression metrics (RMSE, MAE, R2, Pearson r, Spearman rho).
   - Implements `project_disjoint_split` and `project_disjoint_cv` to enforce zero data leakage.
   - Defines `calculate_novelty_score` and `get_novelty_level`.
5. `src/api/service.py`:
   - Implements `NoveltyService`, a memory-efficient singleton that loads the fitted model artifacts once at server startup and exposes the `score_proposal` method.
6. `scripts/clean_dataset.py`:
   - Audits the raw pairwise dataset, detects exact and reversed duplicate pairs, checks label consistency, and produces `similarity_pairs_clean.csv`.
7. `scripts/optimize_models.py`:
   - Executes 5-fold project-disjoint cross-validation across Ridge regression alphas and gradient boosting to determine the optimal model configuration.
8. `scripts/evaluate_holdout.py`:
   - Evaluates the selected Ridge model on the test partition and records predictions and metrics.

9. `scripts/score_novelty.py`:

- Command-line interface allowing users and automated testing scripts to evaluate a proposal from a JSON file or terminal flags.

2.3 Exact System Execution Lifecycle

The system operates across three distinct lifecycles:

PHASE 1: OFFLINE DATA PREPARATION & TRAINING LIFECYCLE

Step 1: Raw Excel dataset (`Academic_Project_Similarity_Synthetic_Dataset.xlsx`) is loaded by `clean_dataset.py`.

Step 2: Four redundant pairs are removed; 2,996 clean pairs are saved to `similarity_pairs_clean.csv`.

Step 3: `optimize_models.py` executes 5-fold project-disjoint cross-validation.

Step 4: The 600 projects are fitted into eight `TfidfVectorizer` instances using `SectionTFIDFExtractor`.

Step 5: The Ridge regression model is fitted on training pair similarities and serialized to `models/ridge_fusion_model.joblib`.

Step 6: The fitted `SectionTFIDFExtractor` is serialized to `models/section_tfidf_extractor.joblib`.

PHASE 2: SERVER STARTUP & MODEL LOADING LIFECYCLE

Step 1: FastAPI server starts (`uvicorn src.api.main:app`).

Step 2: `get_novelty_service()` in `src/api/service.py` is invoked.

Step 3: `models/section_tfidf_extractor.joblib` and `models/ridge_fusion_model.joblib` are loaded into memory once.

Step 4: `NoveltyExplainer` is initialized and cached in memory.

Step 5: SQLite database `data/proposals.db` initializes its schema (`submitted_proposals` table).

PHASE 3: RUNTIME PROPOSAL EVALUATION LIFECYCLE

Step 1: User fills the 8 proposal fields in the React frontend and clicks "Evaluate Novelty".

Step 2: Frontend sends an HTTP POST request to `/api/score`.

Step 3: FastAPI validates the payload using Pydantic schema `ProposalInput` in `schemas.py`.

Step 4: `NoveltyService.score_proposal()` executes:

- Transforms the 8 candidate text sections using the 8 pre-fitted TF-IDF vectorizers.
- Computes cosine similarity against all 600 reference projects.
- Multiplies similarities by Ridge coefficients and adds the intercept.
- Applies self-exclusion rules (filtering out exact ID, title, or identical records).
- Sorts historical projects by predicted similarity descending.
- Generates XAI evidence (snippets and shared keywords) for the top 5 matches.

Step 5: API returns the complete XAI JSON payload to the frontend.

Step 6: React renders the novelty gauge, Recharts bar charts, and expandable match cards.

Step 7: The candidate proposal is NOT saved to the database during scoring.

Step 8: If the user explicitly clicks "Save Proposal", a separate HTTP POST to `/api/save` stores the proposal and its evaluation report in `data/proposals.db`.

Chapter 3: Dataset Explanation & Schema Inspection

CHAPTER 3: DATASET EXPLANATION & SCHEMA INSPECTION

3.1 Raw Dataset Overview: Academic_Project_Similarity_Synthetic_Dataset.xlsx

The system's training and baseline comparison data resides in `data/raw/Academic_Project_Similarity_Synthetic_Dataset.xlsx`. This Microsoft Excel workbook consists of four distinct sheets:

1. `Projects`: Contains 600 unique historical academic project records.

2. `SimilarityPairs`: Contains 3,000 labeled pairwise comparisons between projects.

3. `DataDictionary`: Documents field definitions, valid ranges, and source sheets.

4. `README`: Describes dataset provenance, purpose, and synthetic limitations.

3.2 Inspection of the `Projects` Sheet (600 Rows, 16 Columns)

The `Projects` sheet defines the reference universe of historical capstone projects. It contains 600 rows and 16 columns:

Field Name	Data Type (Python)	Description / Role
project_id	string (object)	Unique primary key (format: "SYN-0001" to "SYN-0600").
academic_year	string (object)	Academic cohort year (e.g., "2022-23", "2023-24").
department	string (object)	Academic unit (e.g., "Computer Science", "IT", "MCA").
program_level	string (object)	Student tier: "UG Mini Project", "UG Mega Project", "PG Diss".
project_status	string (object)	Operational status (all marked "Completed").
domain	string (object)	Technical domain (12 distinct academic disciplines).
project_title	string (object)	[SECTION 1] Project title.
abstract	string (object)	[SECTION 2] Comprehensive summary abstract.
problem_statement	string (object)	[SECTION 3] Domain challenge and motivation.
objectives	string (object)	[SECTION 4] Numbered technical research goals.
proposed_methodology	string (object)	[SECTION 5] Architecture, algorithms, and dataflow.

dataset	string (object)	[SECTION 6] Datasets and benchmarks used.
algorithm_model	string (object)	[SECTION 7] Primary machine learning/algorithmic models.
technology_tools	string (object)	[SECTION 8] Software libraries, tools, and frameworks.
keywords	string (object)	Comma-separated domain keywords.
data_source	string (object)	Provenance flag: 100% of rows contain "SYNTHETIC".

Program Level Distribution (Exactly balanced):

- UG Mini Projects: 200 records (33.33%)
- UG Mega Projects: 200 records (33.33%)
- PG Dissertations: 200 records (33.33%)

Domain Coverage (12 Domains):

- Artificial Intelligence & Machine Learning
- Computer Vision
- Natural Language Processing
- Internet of Things (IoT)
- Cybersecurity
- Blockchain
- Cloud Computing
- FinTech
- Healthcare & Medical Informatics
- Smart Agriculture
- Education Technology
- Computer Networks & Software Defined Networking (SDN)

3.3 Inspection of the `SimilarityPairs` Sheet (3,000 Rows, 8 Columns)

The `SimilarityPairs` sheet contains the training targets for the regression model:

Field Name	Data Type (Python)	Description / Role
pair_id	string (object)	Unique identifier (format: "PAIR-0001" to "PAIR-3000").
project_A_id	string (object)	Foreign key referencing project_id of Project A.
project_B_id	string (object)	Foreign key referencing project_id of Project B.
same_semantic_family	integer (int64)	Binary flag: 1 if projects share a domain/topic, 0 otherwise.

baseline_lexical_overlap_proxy	float (float64)	Continuous synthetic proxy of word overlap [0.6364 to 1.0].
expert_similarity_label	integer (int64)	REGRESSION TARGET: Integer similarity label in [0, 4].
similarity_category	string (object)	Human-readable tier name (e.g., "Not Similar", "Highly Sim").
annotation_note	string (object)	Metadata disclosure: "SYNTHETIC BENCHMARK LABEL...".

Target Similarity Label Distribution (Before vs. After Cleaning):

- Label 0 ("Not Similar"): 1,336 original rows (44.53%) -> 1,332 clean rows (44.46%)
- Label 1 ("Slightly Similar"): 334 rows (11.13%) -> 334 clean rows (11.15%)
- Label 2 ("Moderately Similar"): 514 rows (17.13%) -> 514 clean rows (17.16%)
- Label 3 ("Highly Similar"): 581 rows (19.37%) -> 581 clean rows (19.39%)
- Label 4 ("Nearly Duplicate"): 235 rows (7.83%) -> 235 clean rows (7.84%)

Total: 3,000 original pairs -> Exactly 2,996 clean pairs.

3.4 Audit of Dataset Deduplication (Removal of 4 Redundant Rows)

During data cleaning (`scripts/clean_dataset.py`), the 3,000 pairs were audited for duplicate relationships:

1. PAIR-1819 (SYN-0524 vs SYN-0451, Label 0): Removed as an exact duplicate of PAIR-1324.

2. PAIR-2023 (SYN-0130 vs SYN-0097, Label 0): Removed as a reversed duplicate of PAIR-0432 (SYN-0097 vs SYN-0130).

3. PAIR-2195 (SYN-0165 vs SYN-0249, Label 0): Removed as a reversed duplicate of PAIR-1401 (SYN-0249 vs SYN-0165).

4. PAIR-2957 (SYN-0538 vs SYN-0174, Label 0): Removed as a reversed duplicate of PAIR-2588 (SYN-0174 vs SYN-0538).

Concordance Check: In all four duplicate instances, both counterparts carried identical label 0 annotations and matching proxy scores. No contradictory annotations existed. The cleaned dataset of 2,996 rows is saved in `data/processed/similarity_pairs_clean.csv`.

3.5 Ground Truth Disclosure: Synthetic Nature & Faculty Status

CRITICAL VIVA / REPORT DISCLOSURE:

- 100% of the similarity labels are SYNTHETIC. The README explicitly states:

```
"Synthetic labels are NOT ground truth. For publication, a representative subset must be
independently annotated by faculty/domain experts..."
```

- All 3,000 pairs have the note: "SYNTHETIC BENCHMARK LABEL — must be replaced/validated by faculty annotation before publication."
- An inspection of `data/raw/Faculty_Verification_DomainWise.xlsx` confirms that the columns `Faculty Verdict (0-4)`, `Faculty Comments`, and `Verified By` contain ZERO (0) non-null values across all 13 sheets.
- CONCLUSION: The machine learning model was trained to emulate a synthetic labeling rule, NOT real human academic expert judgment.

3.6 Training Dataset vs. Runtime Reference Corpus

It is critical to distinguish how data is used during training versus runtime:

- Training Dataset: The 2,996 pairwise comparisons in `data/processed/similarity_pairs_clean.csv` (using 480 training projects) were used to train the Ridge regression model weights.
- Runtime Reference Corpus: The 600 project records from the `Projects` sheet are serialized inside `models/section_tfidf_extractor.joblib`. When a user submits a new proposal at runtime, the candidate proposal is compared against these 600 fixed baseline projects.
- Separation of User Submissions: Newly evaluated user proposals are saved into SQLite (`data/proposals.db`). They DO NOT enter or contaminate the 600 baseline projects in the reference corpus.

3.7 The Eight Proposal Sections & Their Individual Contributions

1. `project_title`: Captures high-level focus (e.g., "Crop Disease Detection using CNN"). Short text (average 8-12 words).

2. `abstract`: Comprehensive summary containing technical context, primary dataset, algorithms, and expected performance.

3. `problem_statement`: Highlights the domain deficiency or literature gap motivating the research.

4. `objectives`: Outlines the technical milestones and deliverables.

5. `proposed_methodology`: Detailed operational workflow, mathematical pipelines, and architecture diagrams described in text.

6. `dataset`: The named data resources. IN THIS MODEL, THIS IS THE MOST SENSITIVE SECTION.

7. `algorithm_model`: Specific ML models, neural architectures, or algorithmic techniques.

8. `technology_tools`: Libraries, programming frameworks, and hardware environments.

Chapter 4: How Training Data Is Prepared

CHAPTER 4: HOW TRAINING DATA IS PREPARED

4.1 The Complete Data Preparation Pipeline

The transformation from raw Excel workbooks into machine-readable feature matrices proceeds through five verifiable steps:

Raw Excel Files -> Data Loading & Validation -> Deduplication -> Project-Disjoint Splitting -> Pairwise Feature Extraction

4.2 Step 1: Loading Raw Data (`src/data/loader.py`)

Data loading is managed by three functions in `src/data/loader.py`:

- `load_projects(filepath)`: Loads the `Projects` sheet using `pandas.read_excel`. Validates that all 16 required columns exist and that `project_id` values are unique non-empty strings. Missing text fields are imputed with empty strings (`""`).
- `load_similarity_pairs(filepath)`: Loads the `SimilarityPairs` sheet (or clean CSV). Verifies required keys (`pair_id`, `project_A_id`, `project_B_id`, `expert_similarity_label`), strips whitespace from identifiers, and casts labels to integers.
- `load_feature_csv(filepath)`: Loads pre-extracted feature tables, validating that feature columns are numeric floats.

4.3 Step 2: Cleaning and Deduplication (`scripts/clean_dataset.py`)

The raw 3,000 pairs contain undirected comparisons where project order does not change semantic similarity. `clean_dataset.py` constructs a canonical undirected key for every pair:

```
canonical_key = (min(proj_A, proj_B), max(proj_A, proj_B))
```

- Rows sharing the same `canonical_key` are flagged as duplicates.
- Exactly one exact duplicate (`PAIR-1819`) and three reversed duplicates (`PAIR-2023`, `PAIR-2195`, `PAIR-2957`) were dropped.
- The remaining 2,996 clean rows were exported to `data/processed/similarity_pairs_clean.csv`.

4.4 Step 3: Project-Disjoint Partitioning (`src/utils/metrics.py`)

Standard random splitting of pair rows causes severe data leakage: if Project SYN-0010 appears in a training pair (SYN-0010 vs SYN-0025) and in a test pair (SYN-0010 vs SYN-0080), the model has already memorized the vocabulary and features of SYN-0010 during training!

To prevent data leakage, `project_disjoint_split` partitions the dataset by UNIQUE PROJECTS, not pair rows:

1. Extract all 600 unique project IDs.

2. Shuffle project IDs using a fixed seed (`seed=42`).

3. Partition project IDs into 80% training projects (480 projects) and 20% test projects (120 projects).

4. Train Pairs: Kept ONLY if BOTH `project_A_id` AND `project_B_id` belong to the 480 training projects (yielding 1,916 pairs).

5. Test Pairs: Kept ONLY if BOTH `project_A_id` AND `project_B_id` belong to the 120 test projects (yielding 120 pairs).

6. Cross-Boundary Pairs: Any pair where one project is in Train and the other is in Test (960 pairs) is DISCARDED entirely from evaluation.

Verification:

- Train Pairs: 1,916
- Test Pairs: 120
- Cross-Boundary Pairs Discarded: 960
- Total: 1,916 + 120 + 960 = 2,996 clean pairs.
- Leakage status: ZERO project overlap across train and test partitions.

4.5 Step 4: Constructing Pairwise Features (`src/features/section_tfidf.py`)

```
For each pair $(A, B)$ in the training set, the model does NOT simply concatenate raw text. Instead, it extracts an eight-dimensional feature vector $\vec{x} = [s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8]^T$, where each $s_i$ represents the cosine similarity between the TF-IDF vectors of Section $i$:
```

```
s_i = cosine_similarity( v_A^{(i)}, v_B^{(i)} )
```

Where:

- $v_A^{(i)}$ is the TF-IDF representation of Project A in Section i.
- $v_B^{(i)}$ is the TF-IDF representation of Project B in Section i.
- The 8 section features are:

```
1. `title_similarity`
2. `abstract_similarity`
3. `problem_similarity`
4. `objectives_similarity`
5. `methodology_similarity`
6. `dataset_similarity`
7. `algorithm_similarity`
8. `technology_similarity`
```

4.6 Step 5: Understanding the Synthetic Label Generation Logic

Analysis of `data/raw/Academic_Project_Similarity_Synthetic_Dataset.xlsx` reveals the deterministic generation rules:

- When `same_semantic_family == 0` (projects from different domains):

```
The synthetic generator assigned `expert_similarity_label = 0` ("Not Similar").
```

- When `same_semantic_family == 1` (projects within the same domain/family):

```
The synthetic generator assigned labels 1, 2, 3, or 4 based on token overlap templates:
```

- Label 1: Partial overlap in generic tools.
- Label 2: Overlap in problem domain and methodology.
- Label 3: Overlap in problem, methodology, and dataset.
- Label 4: Near-identical dataset, algorithm, and abstract.

4.7 Concrete Before-and-After Example

BEFORE PREPARATION (Raw Text Pair):

- Project A (SYN-0001):
- `project_title`: "Prototype Crop Disease Detection Using CNN"
- `dataset`: "PlantVillage"
- `algorithm_model`: "CNN"
- Project B (SYN-0157):

- `project_title`: "AI-Assisted Crop Disease Detection Using CNN"
- `dataset`: "PlantVillage"
- `algorithm_model`: "CNN"
- Ground Truth Label: 3 ("Highly Similar")

AFTER PREPARATION (Feature Vector Row):

- `title_similarity`: 0.6131
- `abstract_similarity`: 1.0000
- `problem_similarity`: 1.0000
- `objectives_similarity`: 1.0000
- `methodology_similarity`: 0.0613
- `dataset_similarity`: 1.0000
- `algorithm_similarity`: 1.0000
- `technology_similarity`: 0.0425
- Target `y`: 3.0

Chapter 5: What the ML Model Learns

CHAPTER 5: WHAT THE MACHINE LEARNING MODEL LEARNS

5.1 The Two-Stage Architecture Overview

The system uses a two-stage hybrid machine learning architecture:

- Stage 1 (Lexical Representation): Section-wise TF-IDF (Term Frequency-Inverse Document Frequency) vectorization with unigram and bigram n-grams and stop-word filtering.
- Stage 2 (Multivariate Fusion & Calibration): L2-regularized Ridge Regression that takes the eight section cosine similarities as input and predicts an overall similarity score on a continuous [0.0, 4.0] scale.

5.2 Deep Dive into TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF is an established natural language processing technique that converts text strings into sparse numerical vectors reflecting the statistical importance of words.

Key Definitions for Beginners:

- Token: A single word or sub-word unit extracted from text (e.g., "convolutional", "neural", "plantvillage").
- Document: In this system, each individual section of a project is treated as a separate document (e.g., the `abstract` of Project SYN-0001 is Document 1 for the Abstract Vectorizer).
- Corpus: The collection of all 600 documents for that section across the entire reference dataset.
- Vocabulary: The set of all unique recognized unigrams and bigrams learned by the vectorizer during fitting.

Mathematical Formulation (as implemented in scikit-learn):

1. Term Frequency ($TF(t, d)$):

Measures how frequently term t appears in document d:

```
TF(t, d) = count of term t in document d
```

2. Inverse Document Frequency ($IDF(t, D)$):

Measures how rare or informative term t is across all N documents in the corpus:

```
IDF(t) = ln( (1 + N) / (1 + DF(t)) ) + 1.0
```

Where:

- $N = 600$ (total number of projects in the reference corpus).
- $DF(t)$ is the document frequency (the number of projects whose section contains term t).
- The `+1` terms represent Laplace smoothing (`smooth_idf=True`), preventing division by zero.

Interpretation: Words appearing in almost every project (e.g., "model", "system") receive a low IDF weight. Rare technical terms appearing in only three projects (e.g., "plantvillage", "vit") receive a high IDF weight.

3. Raw TF-IDF Weight:

```
raw_tfidf(t, d) = TF(t, d) * IDF(t)
```

4. L2 Normalization (Euclidean Length Normalization):

Each document vector is divided by its Euclidean norm (L_2 norm) so that longer texts do not produce artificially inflated cosine similarities:

```
v_norm = v / sqrt( sum( v_i^2 ) )
```

5. N-Grams: Unigrams vs. Bigrams:

The configuration specifies `ngram_range=(1, 2)`.

- Unigram (1-gram): Single words (e.g., "neural", "network").
- Bigram (2-gram): Contiguous sequences of two words (e.g., "neural network", "disease detection").

Bigrams capture contextual domain phrasing that unigrams lose.

5.3 Exact Learned TF-IDF Parameters & Vocabulary Sizes

Inspection of `models/section_tfidf_extractor.joblib` reveals the exact vocabulary sizes learned by the 8 vectorizers:

Section Key	Source Column	N-Gram Range	Vocabulary Size
title	project_title	(1, 2)	477 n-grams
abstract	abstract	(1, 2)	454 n-grams
problem	problem_statement	(1, 2)	199 n-grams
objectives	objectives	(1, 2)	188 n-grams
methodology	proposed_method	(1, 2)	227 n-grams
dataset	dataset	(1, 2)	83 n-grams
algorithm	algorithm_model	(1, 2)	19 n-grams
technology	technology_tools	(1, 2)	89 n-grams
Total Combined	All 8 Sections		1,736 n-grams

Stop Words Configuration:

All eight vectorizers use scikit-learn's built-in English stop-word dictionary (318 common English filler words such as "the", "is", "at", "which").

5.4 Deep Dive into Ridge Regression (L2-Regularized Linear Fusion)

Why Ridge Regression?

Why not use standard Ordinary Least Squares (OLS) or a Deep Neural Network?

1. Interpretability: A linear model produces exact, human-understandable additive weights for each section. If the predicted similarity is 2.12, the system can prove that 1.93 came from the Dataset, 0.20 came from the Abstract, and so forth. Deep neural networks are opaque black boxes.

2. Multicollinearity: The eight section similarities are correlated (projects sharing an abstract often share methodology and algorithms). Standard OLS regression becomes unstable under correlated features. Ridge regression stabilizes weights using L2 regularization.

Mathematical Objective of Ridge Regression:

Given training feature matrix $X \in \mathbb{R}^{n imes 8}$ and target similarity labels $y \in [0, 4]^n$, Ridge regression solves:

```
min_w { || y - (Xw + b) ||_2^2 + alpha * || w ||_2^2 }
```

Where:

- $|| y - (Xw + b) ||_2^2 = \sum_{j=1}^n (y_j - (\sum_{i=1}^8 w_i x_{ji} + b))^2$ is the sum of squared prediction errors.

- $alpha * || w ||_2^2 = alpha * \sum_{i=1}^8 w_i^2$ is the L2 penalty that shrinks coefficients toward zero to prevent overfitting.
- Closed-form solution:

```
w = (X^T X + alpha * I)^{-1} X^T (y - mean(y))
```

5.5 Actual Fitted Model Parameters

Inspection of `models/ridge_fusion_model.joblib` reveals:

- Model Class: `sklearn.linear_model.Ridge`
- Regularization Parameter: `alpha = 1.0`
- Intercept (b or $■eta_0$): `-0.07063171692234005`
- Number of Features: Exactly 8 (one per proposal section)
- Fitted Coefficients (w_i):
- `title_similarity`: +0.13404783
- `abstract_similarity`: +0.39804354
- `problem_similarity`: -0.12586323
- `objectives_similarity`: +0.23673393
- `methodology_similarity`: +0.00989785
- `dataset_similarity`: +2.20544639
- `algorithm_similarity`: -0.06507632
- `technology_similarity`: -0.14564642
- Sum of Coefficients ($\sum w_i$): +2.64758357 (~ +2.6476)

Chapter 6: How the Eight Sections Are Combined

CHAPTER 6: HOW THE EIGHT SECTIONS ARE COMBINED

6.1 Section Independence & Specialized Vectorizers

A crucial architectural decision in this system is that sections are NOT pooled into a single combined text blob prior to feature extraction.

Why?

In Notebook 02 (`02_tfidf_baseline.ipynb`), all eight sections were concatenated into a single string. This produced poor diagnostic resolution: if two projects shared a few technology words ("Python", "TensorFlow"), the global TF-IDF score could not distinguish whether the overlap occurred in the core scientific methodology or merely in incidental programming tools.

Therefore, in Notebook 03 (`03_section_wise_tfidf.ipynb`) and production (`src/features/section_tfidf.py`), eight independent `TfidfVectorizer` instances are maintained:

- Vectorizer 1: Fitted ONLY on `project_title` texts.
- Vectorizer 2: Fitted ONLY on `abstract` texts.
- Vectorizer 3: Fitted ONLY on `problem_statement` texts.
- Vectorizer 4: Fitted ONLY on `objectives` texts.
- Vectorizer 5: Fitted ONLY on `proposed_methodology` texts.
- Vectorizer 6: Fitted ONLY on `dataset` texts.
- Vectorizer 7: Fitted ONLY on `algorithm_model` texts.
- Vectorizer 8: Fitted ONLY on `technology_tools` texts.

6.2 Computing Section-by-Section Cosine Similarities

When comparing Project A and Project B, the system transforms each text section into an L2-normalized TF-IDF vector and computes cosine similarity:

```
s_i = cos( v_A^{(i)}, v_B^{(i)} ) = ( v_A^{(i)} . v_B^{(i)} ) / ( ||v_A^{(i)}|| * ||v_B^{(i)}|| )
```

```
Because vectors are already L2-normalized ($||v|| = 1.0$), this simplifies to the dot product:
```

```
s_i = sum_k ( v_{A,k}^{(i)} * v_{B,k}^{(i)} )
```

Each section similarity s_i is bounded in the interval $[0.0, 1.0]$:

- $s_i = 1.0$: Identical lexical vocabulary and proportional frequency in that section.
- $s_i = 0.0$: Zero shared vocabulary (completely disjoint n-grams).

6.3 The 8-Dimensional Feature Representation

The eight scalar section similarities are concatenated into a single feature vector $\vec{x} \in [0.0, 1.0]^8$:

```
\vec{x} = [ s_title, s_abstract, s_problem, s_objectives, s_methodology, s_dataset, s_algorithm,
s_technology ]^T
```

6.4 Distinction: Manually Configured vs. Machine-Learned Section Weights

```
In literature and simple heuristic projects, researchers often assign ad-hoc manual weights (e.g.,
Title = 10%, Abstract = 30%, Dataset = 20%, etc.).
```

In this system:

- NO MANUAL WEIGHTS ARE USED IN SCORING.
- The weights are LEARNED REGRESSION COEFFICIENTS (w_i) computed by scikit-learn's `Ridge(alpha=1.0)` on the training dataset.
- The model decided its own weights based entirely on which section similarities best minimized prediction error against the target labels.

Comparison of Weight Concepts:

Weight Type	Origin and Function
Manually Assigned	Fixed constants chosen by human intuition (e.g., w_title = 0.15).
Weight	NOT USED in this system's scoring engine.
Learned Ridge	Statistically optimized coefficients (w_i) learned by Ridge regression
Coefficient	to minimize squared error on pairwise labels. (e.g., w_dataset = +2.2054).
Signed Feature	The product of a specific section's cosine similarity and its Ridge weight:
Contribution	contribution_i = s_i * w_i. Used in Explainable AI (XAI) reports.

Chapter 7: Training Process, Data Splits & Cross-Validation

CHAPTER 7: TRAINING PROCESS, DATA SPLITS & CROSS-VALIDATION

7.1 Labeled Data Scale

- Total unique projects in reference database: 600 projects (`SYN-0001` to `SYN-0600`).
- Total cleaned pairwise comparisons: 2,996 pairs (after deduplication).
- Target variable: `expert_similarity_label` $\in \{0, 1, 2, 3, 4\}$.

7.2 The Project-Disjoint Partitioning Protocol

The most vital principle in pairwise similarity modeling is avoiding project identity leakage. If an experiment randomly splits the 2,996 pair rows into 80% train and 20% test:

- Project SYN-0015 will appear in the training pairs (e.g., SYN-0015 vs SYN-0040).
- Project SYN-0015 will ALSO appear in the test pairs (e.g., SYN-0015 vs SYN-0112).
- The model can memorize specific project vocabulary and characteristics during training, creating falsely optimistic evaluation metrics.

To ensure scientific honesty, `scripts/optimize_models.py` and `scripts/evaluate_holdout.py` implement `project_disjoint_split` in `src/utils/metrics.py`:

1. The 600 unique projects are randomly shuffled using `np.random.RandomState(42)`.
2. The projects are partitioned into:
   - Training Projects: 480 projects (80.0%)
   - Test Projects: 120 projects (20.0%)
3. Every pair in the dataset is filtered:
   - Train Pairs: Both Project A AND Project B must be in the 480 training projects.

```
Result: Exactly 1,916 pairs.
```

   - Test Pairs: Both Project A AND Project B must be in the 120 test projects.

```
Result: Exactly 120 pairs.
```

   - Cross-Boundary Pairs: Pairs where one project is in Train and the other is in Test are DISCARDED.

```
Result: Exactly 960 pairs discarded.
```

4. Total accountability: $1,916 + 120 + 960 = 2,996$ clean pairs.

7.3 Leakage-Free TF-IDF Fitting

In rigorous machine learning, feature extractors must be fitted ONLY on training documents.

Auditing `scripts/evaluate_holdout.py` (lines 114–135) confirms:

- `TfidfVectorizer.fit()` is called ONLY on the 480 training projects.
- Vocabulary learned on `project_title`: exactly 450 n-grams.
- Exactly 27 technical n-grams that occur exclusively in the 120 test projects were omitted from the vocabulary.
- Zero vocabulary or inverse document frequency leakage occurred from test projects into the feature pipeline.

7.4 Five-Fold Project-Disjoint Cross-Validation

In `scripts/optimize_models.py`, hyperparameter optimization was conducted using 5-fold project-disjoint cross-validation:

- The 600 project IDs were split into 5 disjoint folds of 120 projects each (`n_splits=5`, `seed=42`).
- In each fold, 4 folds (480 projects, ~1,915 pairs) formed the training set; 1 fold (120 projects, ~117 pairs) formed the validation set.
- Evaluated candidate alphas for Ridge regression: $■lpha \in [0.01, 0.1, 1.0, 10.0, 100.0]$.
- Cross-validation results across alphas:
- $■lpha = 0.01$: Mean RMSE = 0.6558, Mean MAE = 0.4770, Mean R2 = 0.7705
- $■lpha = 0.1$: Mean RMSE = 0.6557, Mean MAE = 0.4768, Mean R2 = 0.7706
- $■lpha = 1.0$: Mean RMSE = 0.6548, Mean MAE = 0.4761, Mean R2 = 0.7712
- $■lpha = 10.0$: Mean RMSE = 0.6511, Mean MAE = 0.4731, Mean R2 = 0.7737 (Best CV)
- $■lpha = 100.0$: Mean RMSE = 0.6703, Mean MAE = 0.4901, Mean R2 = 0.7602

7.5 Audit Disclosure: The Relationship Between the Holdout and Fold 5

As documented in the scientific audit (`results/validation/scientific_reliability_audit.md`):

- In `scripts/evaluate_holdout.py`, the 80/20 train/test project partition used `RandomState(42)`.
- In `scripts/optimize_models.py`, `KFold(n_splits=5, shuffle=True, random_state=42)` partitioned the identical sorted list of projects.
- Under scikit-learn's `KFold`, Fold 5 assigns the exact same project index slice (`480:600`) as the test split!
- Therefore, the holdout test set is mathematically identical to Fold 5 of the cross-validation.
- Because $■lpha = 10.0$ was selected using the 5-fold average (which included Fold 5), the holdout partition was NOT an untouched holdout in the strictest machine learning sense. It had indirect exposure during model selection.
- Furthermore, in the final production artifact `models/ridge_fusion_model.joblib`, the saved model was trained with `alpha=1.0`.

7.6 Per-Fold Performance Breakdown (Why Holdout RMSE is 0.7734)

The holdout evaluation reported $ ext{RMSE} = 0.7734$, which is higher than the 5-fold mean ($ ext{RMSE} = 0.6511$).

Inspecting `results/model_optimization/fold_results.csv` explains this discrepancy completely:

Fold	Train Pairs Count	Held-out Pairs	RMSE	MAE	R2 Score	Spearman rho
Fold 1	1,876 pairs	117 pairs	0.5854	0.4524	0.8351	0.8195
Fold 2	1,940 pairs	100 pairs	0.6035	0.4124	0.8040	0.8130
Fold 3	1,908 pairs	127 pairs	0.5707	0.4133	0.8213	0.8305
Fold 4	1,935 pairs	123 pairs	0.7227	0.5223	0.7402	0.7501
Fold 5	1,916 pairs	120 pairs	0.7734	0.5648	0.6677	0.7508
Mean	1,915 pairs	117.4 pairs	0.6511	0.4731	0.7737	0.7928
Std Dev			+-0.0910	+-0.0680	+-0.0695	+-0.0391

Conclusion: Fold 5 happens to be the most challenging partition in the dataset (1.34σ above the mean). The holdout test result (0.7734) reflects evaluating on Fold 5.

Chapter 8: All Calculations and Mathematical Formulas

CHAPTER 8: ALL CALCULATIONS AND MATHEMATICAL FORMULAS

8.1 Complete Mathematical Inventory

Every calculation executed by the system from text input to the final novelty display is documented below with exact formulas.

8.2 Formula 1: TF-IDF Vectorization

For each section $k \in \{1, \dots, 8\}$ and term t in text d:

```
TF(t, d) = count(t, d)
IDF(t) = ln( (1 + 600) / (1 + DF(t)) ) + 1.0
w_raw(t, d) = TF(t, d) * IDF(t)
v_k = w_raw / sqrt( sum_j (w_raw_j^2) )
```

8.3 Formula 2: Section-Level Cosine Similarity

Between candidate vector $v_{cand}^{(k)}$ and reference vector $v_{ref}^{(k)}$:

```
s_k = v_{cand}^{(k)} . v_{ref}^{(k)} = sum_j ( v_{cand, j}^{(k)} * v_{ref, j}^{(k)} )
```

Where $s_k \in [0.0, 1.0]$.

8.4 Formula 3: Additive Ridge Regression Raw Prediction

```
y_raw = beta_0 + sum_{k=1}^8 ( w_k * s_k )
```

Using the exact fitted model parameters from `models/ridge_fusion_model.joblib`:

```
y_raw = -0.07063172
+ (0.13404783 * s_title)
+ (0.39804354 * s_abstract)
```

• (0.12586323 * s_problem)

```
+ (0.23673393 * s_objectives)
+ (0.00989785 * s_methodology)
+ (2.20544639 * s_dataset)
```

• (0.06507632 * s_algorithm)

• (0.14564642 * s_technology)

8.5 Formula 4: Section Signed Model Contribution (XAI)

For each section k:

```
contrib_k = w_k * s_k
```

Reconstruction Invariant (checked in unit tests):

```
y_raw == beta_0 + sum_{k=1}^8 contrib_k (within 1e-5 floating point tolerance)
```

8.6 Formula 5: Prediction Bounding / Clipping

Because linear regression is unconstrained and can theoretically output values < 0.0 or > 4.0, predictions are clipped to the valid target interval:

```
y_clipped = clip( y_raw, 0.0, 4.0 )
y_clipped = max( 0.0, min( 4.0, y_raw ) )
```

8.7 Formula 6: Maximum Historical Similarity Selection

Across all M eligible reference corpus projects (strictly excluding candidate self-matches):

```
s_max = max_{j in {1, ..., M}} { y_{clipped, j} }
```

8.8 Formula 7: Estimated Novelty Percentage

In `src/utils/metrics.py` (lines 88–95):

```
sim_norm = clip( s_max / 4.0, 0.0, 1.0 )
Novelty (%) = (1.0 - sim_norm) * 100.0
```

Equivalently:

```
Novelty (%) = 100.0 * ( 1.0 - (s_max / 4.0) )
```

8.9 Why 4.0 Is Used as the Denominator

The denominator `4.0` is NOT an arbitrary constant. It represents the maximum possible ground-truth integer rating in the benchmark scale:

- 0 = Not Similar
- 1 = Slightly Similar
- 2 = Moderately Similar
- 3 = Highly Similar
- 4 = Nearly Duplicate (Identical)

Dividing by 4.0 normalizes the 0-to-4 regression score into a [0.0, 1.0] fractional similarity ratio before subtracting from 1.0 to obtain novelty.

8.10 Analysis of Extreme Values

- When $s_{max} = 0.0$:

```
sim_norm = 0.0 / 4.0 = 0.0 -> Novelty = (1.0 - 0.0) * 100% = 100.0% (Maximum possible novelty).
```

- When $s_{max} = 1.0$:

```
sim_norm = 1.0 / 4.0 = 0.25 -> Novelty = (1.0 - 0.25) * 100% = 75.0%.
```

- When $s_{max} = 2.0$:

```
sim_norm = 2.0 / 4.0 = 0.50 -> Novelty = (1.0 - 0.50) * 100% = 50.0%.
```

- When $s_{max} = 3.0$:

```
sim_norm = 3.0 / 4.0 = 0.75 -> Novelty = (1.0 - 0.75) * 100% = 25.0%.
```

- When $s_{max} = 4.0$:

```
sim_norm = 4.0 / 4.0 = 1.0 -> Novelty = (1.0 - 1.0) * 100% = 0.0% (Zero novelty / complete duplicate).
```

- When $s_{max} < 0.0$ (out-of-range negative prediction):

```
Clipping clamps to 0.0 -> Novelty = 100.0%.
```

- When $s_{max} > 4.0$ (out-of-range positive prediction):

```
Clipping clamps to 4.0 -> Novelty = 0.0%.
```

8.11 Novelty Assessment Tier Thresholds

In `src/utils/metrics.py` (lines 97–107), the novelty percentage is mapped to four standardized qualitative tiers:

Novelty Percentage Range	Exact Assessment Tier String
75.0% <= Novelty <= 100.0%	"High estimated novelty relative to the current database"
50.0% <= Novelty < 75.0%	"Moderate Novelty (Partial Overlap)"
25.0% <= Novelty < 50.0%	"Low Novelty (Significant Prior Work)"
0.0% <= Novelty < 25.0%	"Very Low Novelty / Potential Duplicate"

Rounding and Precision Standards:

- Similarity scores: Rounded to 4 decimal places in API responses (`round(score, 4)`).
- Novelty percentages: Rounded to 2 decimal places in API responses (`round(pct, 2)`) and formatted to 1 decimal place in the UI (`pct.toFixed(1) + "%"`).

Chapter 9: How the Model Decides Its Weights

CHAPTER 9: HOW THE MODEL DECIDES ITS WEIGHTS

9.1 Where Do the Weights Come From?

A common misconception among students preparing for viva is assuming that the developer chose the section weights by hand (e.g., "I decided that the dataset is worth 50% and the title is worth 10%").

In this system:

- THE DEVELOPER DID NOT PICK THE SECTION WEIGHTS.
- The weights were mathematically derived by scikit-learn's Ridge regression algorithm to minimize the squared difference between the model's predictions and the 3,000 ground-truth similarity labels in the training dataset.

9.2 The Mathematical Optimization Process

Ridge regression fits weights $w_1, \dots, w_8$ and intercept b by solving:

```
min_{w, b} sum_{j=1}^{1916} ( y_j - ( sum_{k=1}^8 w_k * s_{jk} + b ) )^2 + alpha * sum_{k=1}^8 w_k^2
```

The optimization considers two competing forces:

1. Fit to Data: Minimize prediction error against the training labels $y_j \in [0, 4]$.

2. L2 Shrinkage Penalty: Penalize large weights ($■lpha \sum w_k^2$) to prevent any single section from taking on an infinite or unstable weight due to feature collinearity.

At $■lpha = 1.0$, the closed-form matrix calculus yields the exact weights stored in `models/ridge_fusion_model.joblib`:

Feature Name	Learned Weight	Qualitative Role in Prediction
Intercept (beta_0)	-0.07063172	Baseline bias constant when all section similarities are 0.
dataset_similarity	+2.20544639	Dominant positive predictor (>55% of total scale).
abstract_similarity	+0.39804354	Strong secondary positive predictor.
objectives_sim	+0.23673393	Moderate positive predictor.
title_similarity	+0.13404783	Modest positive predictor.
methodology_sim	+0.00989785	Negligible positive predictor (~0.01).
algorithm_similarity	-0.06507632	Negative suppressor variable (-0.065).
problem_similarity	-0.12586323	Negative suppressor variable (-0.126).
technology_similarity	-0.14564642	Strongest negative suppressor variable (-0.146).
Sum of Coefficients	+2.64758357	Theoretical maximum total feature contribution when s_i = 1.0

9.3 Why Is the Dataset Weight So Huge (+2.2054)?

Notice that `dataset_similarity` alone contributes up to $+2.2054$ points out of a 4.0 scale!

Why did the machine learning algorithm assign over 55% of its total predictive capacity to the dataset?

- Root Cause: Synthetic Dataset Generation Artifact.
- In `Academic_Project_Similarity_Synthetic_Dataset.xlsx`, whenever pairs were synthesized to have High Similarity (Label 3) or Duplicate Similarity (Label 4), the generator almost always assigned them identical dataset names (e.g., both used "PlantVillage", or both used "BraTS MRI").
- Conversely, whenever pairs had Label 0 ("Not Similar"), their datasets were completely different.
- The Ridge regression algorithm discovered that `dataset_similarity` was the single cleanest, highest-signal predictor of the synthetic label in the training data, so it assigned it a dominant weight of $+2.2054$.

9.4 Why Are Some Weights Negative? The Suppressor Effect

```
It seems counter-intuitive that sharing the same algorithm or technology stack would REDUCE predicted
project similarity ($w_{tech} = -0.1456$, $w_{prob} = -0.1259$, $w_{algo} = -0.0651$).
```

Why did the model learn negative weights?

- In statistics, this is known as a Suppressor Variable Effect caused by Multicollinearity.
- Standard software tools ("Python", "TensorFlow", "Scikit-Learn", "CUDA") and generic algorithm names ("CNN", "SVM", "Random Forest") appear ubiquitously across hundreds of completely dissimilar projects in computer science.
- A project on "Stock Price Prediction" and a project on "Skin Cancer Detection" both list "Python, Scikit-Learn, Pandas" in their technology stack and "Machine Learning / CNN" in their algorithm section.
- If the model had positive weights on technology and algorithms, it would predict an artificially high similarity for completely unrelated projects.
- The Ridge regression model learned that high technology similarity often acts as "noise" or false overlap. To correct for this and prevent false positives across different domains, the optimization assigned small negative weights to algorithms and technologies!

9.5 The Theoretical Ceiling Vulnerability (The 2.5768 Maximum)

The combination of an intercept of -0.0706 and a coefficient sum of $+2.6476$ leads to a severe mathematical limitation:

- Suppose a candidate proposal is an exact 100% duplicate of a historical project across all 8 sections ($s_1 = s_2 = \dots = s_8 = 1.0$).
- What does the Ridge model predict?

```
y_raw = -0.07063172 + (1.0 * 2.64758357) = 2.57695185 (~ 2.5768)
```

- What novelty percentage does this produce?

```
Novelty (%) = 100 * (1.0 - (2.5768 / 4.0)) = 100 * (1.0 - 0.6442) = 35.58%
```

- What tier does 35.58% map to?

```
35.58% in [25.0%, 50.0%) -> "Low Novelty (Significant Prior Work)"
```

- CRITICAL SCIENTIFIC AUDIT FINDING:

```
Even for an exact, 100% identical twin of an existing project, the model prediction can never exceed
2.5768. Therefore, the estimated novelty can never drop below 35.58%. The system can NEVER reach the
"Very Low Novelty / Potential Duplicate" tier (< 25.0%) under its current weights!
```

9.6 Step-by-Step Numerical Example of Feature Contributions

Suppose a candidate proposal is evaluated against historical project SYN-0142:

- `title_similarity` = 0.5000 -> Contrib: 0.5000 * +0.1340 = +0.0670
- `abstract_similarity` = 0.6000 -> Contrib: 0.6000 * +0.3980 = +0.2388
- `problem_similarity` = 0.4000 -> Contrib: 0.4000 * -0.1259 = -0.0504
- `objectives_sim` = 0.7000 -> Contrib: 0.7000 * +0.2367 = +0.1657
- `methodology_sim` = 0.3000 -> Contrib: 0.3000 * +0.0099 = +0.0030
- `dataset_similarity` = 1.0000 -> Contrib: 1.0000 * +2.2054 = +2.2054
- `algorithm_similarity` = 0.8000 -> Contrib: 0.8000 * -0.0651 = -0.0521
- `technology_similarity` = 0.9000 -> Contrib: 0.9000 * -0.1456 = -0.1310

Additive Reconstruction:

Sum of Contributions = +0.0670 + 0.2388 - 0.0504 + 0.1657 + 0.0030 + 2.2054 - 0.0521 - 0.1310

```
= +2.4464
```

Intercept = -0.0706

Raw Predicted Score = -0.0706 + 2.4464 = 2.3758

Clipped Score = 2.3758

```
Novelty Score (%) = 100 * (1.0 - (2.3758 / 4.0)) = 100 * (1.0 - 0.59395) = 40.61% ("Low Novelty")
```

Chapter 10: What Happens When a User Submits a Proposal

CHAPTER 10: WHAT HAPPENS WHEN A USER SUBMITS A PROPOSAL

10.1 End-to-End Runtime Execution Trace

When a student or faculty member uses the React dashboard, the entire request-response cycle executes deterministically through 12 verifiable steps:

Step 1 (Form Input):

The user enters the 8 sections in `frontend/src/components/proposal/ProposalForm.jsx`.

Field validation ensures non-empty strings and minimum character lengths.

Step 2 (API POST Dispatch):

The user clicks "Evaluate Novelty". `frontend/src/services/api.js` dispatches an HTTP POST request to:

`http://127.0.0.1:8000/api/score`

Payload contains JSON: `{ "project_title": "...", "abstract": "...", ... }`.

Step 3 (FastAPI Route Handling):

FastAPI route handler `@app.post("/api/score")` in `src/api/main.py` receives the request.

Pydantic model `ProposalInput` validates field non-emptiness and strip whitespace.

Step 4 (Service Delegation):

The route calls `NoveltyService.score_proposal(candidate_dict, top_k=5)` in `src/api/service.py`.

Step 5 (Vectorization of Candidate Proposal):

The candidate proposal is passed to `NoveltyExplainer.explain_proposal()` in `src/explainability/explainer.py`.

The 8 section texts are transformed by the 8 pre-loaded `TfidfVectorizer` instances:

`v_cand_k = vectorizers[k].transform([candidate_text_k])`

Step 6 (Multi-Project Cosine Matrix Multiplication):

The candidate vector for each section is matrix-multiplied against the precomputed sparse TF-IDF matrix of all 600 reference projects:

`sim_k = cosine_similarity(v_cand_k, tfidf_matrices[k])` -> 600 similarity scores for each section.

Step 7 (Batch Ridge Model Inference):

The resulting $600 imes 8$ feature matrix X is multiplied by the Ridge coefficient vector:

`raw_preds = -0.07063 + (X @ coef_)`

`clipped_preds = np.clip(raw_preds, 0.0, 4.0)`

Step 8 (Candidate Self-Exclusion Filtering):

The candidate proposal is checked against the 600 projects to prevent self-matching:

1. Project ID match: If candidate project_id matches historical project_id.

2. Exact title match: If candidate title matches historical title (case-insensitive).

3. Identical vector match: If all 8 section similarities are >= 0.9999.

Matching records are strictly masked out of the eligible comparison set.

Step 9 (Deterministic Ranking & Top Match Selection):

The remaining eligible projects are sorted:

`sort_values(by=["predicted_similarity", "raw_prediction", "project_id"], ascending=[False, False, True])`

The top 5 closest historical projects are selected.

Step 10 (Novelty Formulation):

The maximum predicted similarity across the corpus is selected:

```
`max_sim = ranked_df.iloc[0]["predicted_similarity"]`
`novelty_pct = 100.0 * (1.0 - (max_sim / 4.0))`
`novelty_tier = get_novelty_level(novelty_pct)`
```

Step 11 (XAI Evidence Generation):

For each of the top 5 matches, `explain_pair()` extracts:

- Section similarities and Ridge signed contributions.
- Shared informative keywords using the fitted vocabulary.
- Word-boundary-snapped source text snippets from candidate and historical records.
- Section difference summary (high overlap >= 0.30 vs. differentiating < 0.15).

Step 12 (JSON Response & UI Presentation):

The API returns `ProposalEvaluationResponse`.

React renders the Novelty Gauge, Recharts bar charts, and expandable match cards.

DATABASE NOTE: At this stage, NO record is written to SQLite.

10.2 Sequence Diagram of Runtime Execution

Frontend		FastAPI /api		NoveltyService		NoveltyExplainer		SQLite DB

```
| | | | | |
| 1. POST /api/score | | | |
|----------------------->| | | |
| | 2. Validate Proposal | | |
| |------------------------->| | |
| | | 3. explain_proposal() | |
| | |--------------------------->| |
| | | | 4. TF-IDF vectorization |
| | | | & cosine sim (600 prj)|
| | | | 5. Ridge inference |
| | | | 6. Self-exclusion filter |
| | | | 7. Top-5 XAI evidence |
| | | 8. Return Report Payload |<-------------------------|
| | |<---------------------------| |
| | 9. HTTP 200 JSON | | |
|<-----------------------|-------------------------| | |
```

```
| | | | |
| [User clicks "Save"] | | | |
| 10. POST /api/save | | | |
|----------------------->| | | |
| | 11. Store in SQLite | | |
| |------------------------------------------------------------------------------->|
| | 12. HTTP 201 Saved | | |
|<-----------------------|-------------------------------------------------------------------------------
-------|
```

Chapter 11: Explainable AI (XAI) Architecture

CHAPTER 11: EXPLAINABLE AI (XAI) ARCHITECTURE

11.1 The Four Layers of Explainability

The explainability engine (`src/explainability/explainer.py`) implements four complementary layers of transparent evidence:

- Layer 1 (Section-Level Rankings): Ranks the 8 sections by raw cosine similarity to show exactly where the overlap is concentrated.
- Layer 2 (Additive Model Attributions): Exposes the exact mathematical Ridge signed contribution for each section, reconstructing the prediction.
- Layer 3 (Shared Informative Keywords): Identifies the exact technical terms and n-grams shared between both proposals, filtered for generic stop words.
- Layer 4 (Source-Verified Text Snippets): Extracts verbatim text windows centered around shared keywords with clean word boundaries.

11.2 Layer 1: Ranked Sections Table

For each match, sections are sorted in descending order of cosine similarity:

`ranked_sections = sorted(sections, key=lambda x: x["raw_similarity"], reverse=True)`

Each entry displays:

- Section Name (e.g., "Dataset", "Abstract", "Algorithm")
- Raw Cosine Similarity (s_k)
- Model Weight (w_k)
- Signed Contribution ($w_k imes s_k$)

11.3 Layer 2: Additive Ridge Feature Attributions

Because Ridge regression is an additive linear model:

```
Predicted Score = Intercept + sum( Signed Contributions )
```

In the dashboard and API report, every prediction is fully decomposable. A faculty evaluator can verify with a pocket calculator that:

```
-0.0706 + sum(contributions) == Predicted Similarity
```

11.4 Layer 3: Informative Keyword Extraction (`extract_shared_terms`)

How does the system identify shared keywords between two sections?

1. Both texts are transformed by the fitted section vectorizer:

`v_cand = vec.transform([cand_text])`

`v_hist = vec.transform([hist_text])`

2. The element-wise Hadamard product of the two normalized vectors is computed:

`overlap = v_cand.multiply(v_hist)`

In cosine similarity, the total score is the sum of these element-wise products: $\sum_k v_1[k] \cdot v_2[k]$.

Each non-zero entry in `overlap` represents an exact n-gram that contributes to the cosine similarity!

3. Stop-Word and Low-Information Filtering (`is_informative_term`):

```
Common generic academic boilerplate terms are filtered out:
`GENERIC_LOW_INFO_TERMS = {"using", "system", "project", "approach", "method", "results", ...}`
```

4. Physical Verification Check:

```
To ensure integrity, the code executes a strict physical regex check:
`rf"(?<!\w){re.escape(term)}(?!\w)"`
The term MUST physically appear as an exact whole word in BOTH source texts!
```

11.5 Layer 4: Source-Verified Snippet Extraction (`extract_snippet`)

To display context around shared keywords without ugly mid-word chops:

1. Word-Boundary Snapping: When cutting a window of 45 characters before and after the keyword, the algorithm snaps outward to the nearest whitespace.

2. Ellipsis Integrity: Ellipses (`...`) are added ONLY if characters were actually omitted before the start or after the end.

3. Verification Check (`verify_snippet`):

```
`return (body in source_text) and (term.lower() in body.lower())`
If the extracted snippet does not exist as an exact substring of the original project text, it is
rejected.
```

11.6 Candidate Self-Exclusion Algorithm

If a student re-submits a proposal that is already indexed in the database (or evaluates an existing project), the system must not compare the project against itself (which would trivially produce similarity 1.0 and 0% novelty).

In `NoveltyExplainer.explain_proposal()`, self-exclusion executes three checks:

1. Project ID Match: If `candidate_id` matches `eligible_df["project_id"]` (case-insensitive).

2. Exact Title Match: If candidate title matches `historical_titles` (case-insensitive).

3. Identical Vector Match: If all 8 section similarities are >= 0.9999.

Matching rows are removed from comparison, ensuring the candidate is only evaluated against *prior peer work*.

Chapter 12: Comprehensive Analysis of Every Notebook & Script

CHAPTER 12: COMPREHENSIVE ANALYSIS OF EVERY NOTEBOOK & SCRIPT

12.1 Detailed Audit of Jupyter Notebooks (`notebooks/`)

NOTEBOOK 01: `notebooks/01_data_exploration.ipynb`

- Purpose: Exploratory data analysis (EDA) of the raw Excel dataset.
- Input Files: `data/raw/Academic_Project_Similarity_Synthetic_Dataset.xlsx`, `data/raw/Project_Similarity_Pairs_0_to_4.xlsx`.
- Output Files: In-memory summary tables and console distributions.
- Main Libraries: `pandas`, `pathlib`.
- Code Summary:
- Inspects sheets: `Projects`, `SimilarityPairs`, `DataDictionary`, `README`.
- Verifies shapes: 600 projects, 3,000 pairs.
- Audits label frequencies: Label 0 (44.5%), Label 1 (11.1%), Label 2 (17.1%), Label 3 (19.4%), Label 4 (7.8%).
- Confirms balanced program levels: exactly 200 UG Mini, 200 UG Mega, 200 PG Dissertations.
- Beginner Takeaway: Understands the raw dataset schema, column types, and the synthetic nature of the labels.

NOTEBOOK 02: `notebooks/02_tfidf_baseline.ipynb`

- Purpose: Evaluates a global concatenated text TF-IDF baseline model.
- Input Files: `Academic_Project_Similarity_Synthetic_Dataset.xlsx`.
- Output Files: `results/tfidf_baseline_correlations.csv`.
- Main Libraries: `pandas`, `sklearn.feature_extraction.text.TfidfVectorizer`, `scipy.stats.spearmanr`.
- Code Summary:
- Concatenates all 8 text fields into a single `combined_text` string per project.
- Fits a global `TfidfVectorizer(stop_words='english', ngram_range=(1,2))`.
- Computes pairwise cosine similarity on `combined_text`.
- Measures correlation against `expert_similarity_label`: Pearson r ~ 0.62, Spearman rho ~ 0.58.
- Beginner Takeaway: Concatenating all text into one blob loses section structure and yields suboptimal correlation.

NOTEBOOK 03: `notebooks/03_section_wise_tfidf.ipynb`

- Purpose: Implements section-disaggregated TF-IDF feature extraction across the 8 sections.
- Input Files: `Academic_Project_Similarity_Synthetic_Dataset.xlsx`.
- Output Files: `results/section_wise_tfidf_features.csv`.
- Main Libraries: `pandas`, `sklearn.feature_extraction.text.TfidfVectorizer`, `sklearn.metrics.pairwise.cosine_similarity`.
- Code Summary:
- Defines section mapping dictionary for 8 fields.
- Fits 8 separate `TfidfVectorizer` instances on individual section columns.
- Computes 8 separate pairwise cosine similarities for all 3,000 pairs.
- Beginner Takeaway: Disaggregating text into 8 distinct section similarities allows individual section importance weighting.

NOTEBOOK 04: `notebooks/04_feature_label_analysis.ipynb`

- Purpose: Analyzes individual section correlations and multicollinearity with ground-truth similarity labels.

- Input Files: `section_wise_tfidf_features.csv`, `Project_Similarity_Pairs_0_to_4.xlsx`.
- Output Files: Correlation heatmaps, feature pair scatter plots.
- Main Libraries: `pandas`, `numpy`, `matplotlib`, `seaborn`, `scipy.stats`.
- Code Summary:
- Computes correlation matrix between the 8 section similarities and the target label.
- Discovers that `dataset_similarity` has the highest correlation with similarity labels (r ~ 0.81).
- Discovers that `problem` and `technology` have high collinearity with `methodology`.
- Beginner Takeaway: Highlights that dataset overlap dominates synthetic labels and identifies collinear features.

NOTEBOOK 05: `notebooks/05_weighted_section_fusion.ipynb`

- Purpose: Experiments with linear section fusion: unweighted average, constrained non-negative optimization, and Ridge regression.
- Input Files: `section_wise_tfidf_features.csv`.
- Output Files: Learned section weight vectors and residual plots.
- Main Libraries: `scipy.optimize.minimize`, `sklearn.linear_model.Ridge`, `sklearn.metrics`.
- Code Summary:
- Evaluates unweighted average: RMSE ~ 0.88.
- Optimizes constrained non-negative weights: RMSE ~ 0.72.
- Fits unconstrained Ridge regression: RMSE ~ 0.65.
- Beginner Takeaway: Shows that Ridge regression achieves the lowest error on the synthetic benchmark.

NOTEBOOK 06: `notebooks/06_dense_semantic_embeddings.ipynb`

- Purpose: Explores dense transformer semantic embeddings (`all-MiniLM-L6-v2`) and hybrid TF-IDF + Dense fusion.
- Input Files: `Academic_Project_Similarity_Synthetic_Dataset.xlsx`.
- Output Files: `models/section_dense_extractor.joblib`, `results/hybrid_model_comparison.csv`.
- Main Libraries: `sentence_transformers`, `torch`, `sklearn.linear_model.Ridge`.
- Code Summary:
- Encodes each of the 8 sections into 384-dimensional dense vectors using `all-MiniLM-L6-v2`.
- Computes 8 dense cosine similarities, forming a 16-feature hybrid matrix (8 TF-IDF + 8 Dense).
- Trains Ridge regression on 16 features: RMSE ~ 0.61.
- Beginner Takeaway: Dense embeddings capture conceptual similarity but are computationally heavier and less transparent than pure TF-IDF.

12.2 Detailed Audit of Training & Evaluation Scripts (`scripts/`)

SCRIPT 1: `scripts/clean_dataset.py`

- Purpose: Audits raw pairs, identifies duplicates, and outputs `data/processed/similarity_pairs_clean.csv`.
- Inputs: `data/raw/Academic_Project_Similarity_Synthetic_Dataset.xlsx`.
- Outputs: `data/processed/similarity_pairs_clean.csv`, `dataset_cleaning_manifest.json`, `dataset_cleaning_report.md`.
- Key Logic: Detects 1 exact duplicate and 3 reversed duplicate pairs using canonical min/max ID tuples.

SCRIPT 2: `scripts/optimize_models.py`

- Purpose: Executes 5-fold project-disjoint cross-validation and hyperparameter tuning for Ridge regression and Gradient Boosting.
- Inputs: `Academic_Project_Similarity_Synthetic_Dataset.xlsx`, `similarity_pairs_clean.csv`.
- Outputs: `results/model_optimization/fold_results.csv`, `summary.csv`, `best_model_config.json`.

- Key Logic: Disjoint KFold splitting on project IDs; fits TF-IDF vectorizers strictly on training projects per fold.

SCRIPT 3: `scripts/evaluate_holdout.py`

- Purpose: Evaluates the selected Ridge model on the test partition.
- Inputs: `Academic_Project_Similarity_Synthetic_Dataset.xlsx`, `similarity_pairs_clean.csv`.
- Outputs: `results/final_evaluation/holdout_metrics.json`, `holdout_predictions.csv`, `final_evaluation_report.md`.
- Key Logic: Evaluates on 120 test pairs using 480 training projects. Note: Test set matches Fold 5 of cross-validation.

SCRIPT 4: `scripts/train.py`

- Purpose: Production pipeline training script that exports fitted checkpoints to `models/`.
- Inputs: `Academic_Project_Similarity_Synthetic_Dataset.xlsx`, `similarity_pairs_clean.csv`.
- Outputs: `models/section_tfidf_extractor.joblib`, `models/ridge_fusion_model.joblib`.
- Key Logic: Fits 8 vectorizers on all 600 projects and trains Ridge regression model on training pairs.

SCRIPT 5: `scripts/score_novelty.py`

- Purpose: Standalone CLI tool to evaluate candidate project proposals from terminal or JSON file.
- Inputs: JSON file or CLI flags (`--title`, `--abstract`, etc.), `models/*.joblib`.
- Outputs: Formatted terminal novelty report or structured JSON.

Chapter 13: Evaluation Metrics & Actual Test Results

CHAPTER 13: EVALUATION METRICS & ACTUAL TEST RESULTS

13.1 Mathematical Definitions of Evaluation Metrics

The regression model's predictive accuracy against ground-truth similarity labels was evaluated using five standard regression and rank correlation metrics:

1. Mean Absolute Error (MAE):

Measures the average magnitude of absolute prediction errors on the [0, 4] scale:

```
MAE = (1 / n) * sum_{j=1}^n | y_j - y_pred_j |
```

Interpretation: An MAE of 0.47 means that on average, the model's similarity prediction deviates from the target label by less than half a point.

2. Root Mean Squared Error (RMSE):

Measures the square root of mean squared errors, penalizing large outlier errors:

```
RMSE = sqrt( (1 / n) * sum_{j=1}^n (y_j - y_pred_j)^2 )
```

3. Coefficient of Determination (R^2 Score):

Measures the proportion of variance in the target labels explained by the model:

```
R^2 = 1.0 - ( sum (y_j - y_pred_j)^2 / sum (y_j - y_mean)^2 )
```

Interpretation: $R^2 = 1.0$ is perfect prediction; $R^2 = 0.0$ indicates performance no better than predicting the mean label.

4. Pearson Correlation Coefficient (r):

Measures linear correlation between true labels and model predictions [-1.0, 1.0].

5. Spearman Rank Correlation Coefficient ($ ho$):

Measures monotonic rank ordering between true labels and model predictions. Vital for retrieval: ensures that more similar projects rank above less similar projects.

13.2 Summary of Actual Reported Results

A. 5-Fold Project-Disjoint Cross-Validation (`results/model_optimization/summary.csv`):

- Average RMSE: 0.6511 +- 0.0910
- Average MAE: 0.4731 +- 0.0680
- Average R2: 0.7737 +- 0.0695
- Average rho: 0.7928 +- 0.0391

B. Final Holdout Evaluation (`results/final_evaluation/holdout_metrics.json`):

- Holdout RMSE: 0.7734
- Holdout MAE: 0.5648
- Holdout R2: 0.6677
- Pearson r: 0.8237
- Spearman rho: 0.7508

Audit Note on Holdout: The holdout metrics are mathematically identical to Fold 5 of cross-validation. Fold 5 was the most challenging fold (the maximum of the distribution), which accounts for why the holdout RMSE (0.7734) is higher

than the 5-fold mean (0.6511).

13.3 Actual Results from the Scientific Reliability Audit

In `tests/test_scientific_reliability.py`, seven controlled cases were evaluated against the production pipeline:

Case #	Test Scenario	Actual Sim	Actual Nov %	Top Retrieved Match	Test Status
1a	Exact copy (no exclude)	2.5768	35.58%	SYN-0157 (Rank #1)	FAIL (1)
				SYN-0001 (Rank #6)	
1b	Exact copy (production)	2.6551	33.62%	SYN-0157	PASS
2	Minor wording edits	2.6352	34.12%	SYN-0157	PASS
3	Paraphrased concepts	0.0175	99.56%	SYN-0156	FAIL (2)
4	Same title, diff body	1.1100	72.25%	SYN-0275	PASS
5	Unrelated quantum topic	0.0000	100.00%	SYN-0001	PASS
6	Single section (dataset)	2.1254	46.87%	SYN-0208	FAIL (3)
7	Generic boilerplate text	0.0985	97.54%	SYN-0378	PASS

Explanation of Audit Failures:

- Failure 1 (Ceiling & Non-Monotonicity): An exact copy scored only 2.5768 (35.58% novelty) and ranked #6 behind peers because negative weights on technology/algorithm penalized the identical match.
- Failure 2 (Paraphrase Blindness): When technical concepts were paraphrased, TF-IDF found zero word overlap, producing a false 99.56% novelty score for a conceptual duplicate.
- Failure 3 (Dataset Dominance): Simply sharing the dataset name "PlantVillage" caused an unrelated marine sensor project to score 2.1254 similarity (46.87% novelty) because the dataset weight is +2.2054.

Chapter 14: Important Limitations & Scientific Validity

CHAPTER 14: IMPORTANT LIMITATIONS & SCIENTIFIC VALIDITY

14.1 The Fundamental Provenance Limitation: Synthetic Ground Truth

The single most critical limitation of this machine learning system is data provenance:

- ALL 3,000 TRAINING PAIR LABELS ARE SYNTHETIC HEURISTIC RULES.
- NOT A SINGLE REAL FACULTY EVALUATION EXISTS IN THE DATASET.
- The high R^2 score (~0.77) does NOT mean the AI understands academic novelty like a professor. It simply proves that the Ridge regression algorithm successfully learned the synthetic formula used to create the labels.
- This system is currently an advanced heuristic prototype, not an empirically validated machine learning classifier.

14.2 Lexical Brittleness vs. Paraphrasing

Because Stage 1 relies on TF-IDF word n-grams:

- If a student rewrites an existing paper using a thesaurus or ChatGPT to replace technical words with synonyms (e.g., changing "convolutional neural network" to "hierarchical spatial feature extractor"), TF-IDF n-gram overlap drops to near zero.
- The model outputs a false "High Novelty" score (> 95%) because it cannot understand abstract semantic meaning.
- Defense: A future version must incorporate dense sentence transformers (`all-MiniLM-L6-v2` or `Specter2`) to detect semantic conceptual similarity regardless of phrasing.

14.3 Disproportionate Sensitivity to Dataset Names

Because `dataset_similarity` has a learned weight of $+2.2054$:

- A proposal that investigates an entirely new mathematical theory will be penalized with a "Low Novelty" rating simply because it tests the theory on a standard open benchmark (like ImageNet or MNIST).
- Conversely, an unoriginal project that repeats an old methodology on a newly named private CSV file will receive an artificially inflated novelty score.

14.4 The Theoretical Ceiling & Non-Monotonicity Vulnerabilities

1. Ceiling Effect: With an intercept of -0.0706 and coefficient sum of $+2.6476$, the maximum possible score is 2.5768 out of 4.0. Novelty cannot drop below 35.58%. The "Very Low Novelty / Duplicate" tier (< 25%) is unreachable.

2. Non-Monotonicity: Negative weights on `technology_tools` (-0.1456), `problem_statement` (-0.1259), and `algorithm_model` (-0.0651) mean that having identical technology or algorithms actually *reduces* predicted similarity. An identical copy can score lower than a near-peer.

3. Solution: The model should be re-fitted using Non-Negative Least Squares (NNLS) to constrain all weights $w_i \ge 0$, and calibrated so that an identical project maps to 4.0 (0% novelty).

14.5 Scientific Roadmap for Legitimate Real-World Validation

To convert this prototype into a published, scientifically validated novelty detection system:

1. Faculty Annotation: Have faculty panels annotate 300 real student project pairs in `data/raw/Faculty_Verification_DomainWise.xlsx`.

2. Inter-Rater Reliability: Measure agreement across faculty annotators (Cohen's Kappa ≥ 0.70).

3. Non-Negative Calibration: Fit constrained regression models ($w_i \ge 0$) calibrated to the $[0.0, 4.0]$ scale.

4. Hybrid Dense Integration: Combine TF-IDF with dense semantic embeddings to defend against paraphrasing.

5. Sealed Holdout Protocol: Reserve a sealed, untouched test split of real projects that is never evaluated until final model freezing.

Chapter 15: End-to-End Worked Numerical Example

CHAPTER 15: END-TO-END WORKED NUMERICAL EXAMPLE

15.1 The Candidate Proposal

Let us walk through the exact mathematical evaluation of a realistic candidate proposal submitted by a student:

- `project_title`: "Federated Learning for Privacy-Preserving Medical Diagnostics"
- `domain`: "Healthcare AI"
- `abstract`: "We propose a federated learning framework using convolutional neural networks to train diagnostic models across distributed hospital datasets without centralizing sensitive patient records."
- `problem_statement`: "Medical imaging datasets are siloed due to strict HIPAA regulations and patient confidentiality concerns."
- `objectives`: "Train high-accuracy classification models across multi-site hospitals while maintaining patient privacy."
- `proposed_methodology`: "Implement FedAvg aggregation with differential privacy noise simulated across multiple client nodes."
- `dataset`: "BraTS multi-institutional magnetic resonance imaging benchmarks."
- `algorithm_model`: "3D U-Net and ResNet-50 Convolutional Neural Networks."
- `technology_tools`: "PyTorch, Flower Federated Framework, Python, CUDA."

15.2 Comparison Against Closest Historical Match (SYN-0142)

The system evaluates the proposal against all 600 reference projects. The closest historical match found in the corpus is:

- Project ID: `SYN-0142`
- Title: "Privacy-Preserving Federated Learning for Multi-Hospital Clinical Diagnostics"
- Domain: "Healthcare & Medical Informatics"

15.3 Step 1: Section-by-Section Cosine Similarity Calculations

The 8 specialized vectorizers compute cosine similarity between the candidate and SYN-0142:

1. `title_similarity` (s_1): 0.5842 (shared: "federated learning", "privacy preserving")
2. `abstract_similarity` (s_2): 0.6410 (shared: "federated learning", "diagnostic", "patient records")
3. `problem_similarity` (s_3): 0.4520 (shared: "patient confidentiality", "regulations")
4. `objectives_similarity` (s_4): 0.5110 (shared: "classification models", "multi site")
5. `methodology_similarity` (s_5): 0.3890 (shared: "fedavg", "aggregation", "differential privacy")
6. `dataset_similarity` (s_6): 1.0000 (shared: "brats", exact benchmark match)
7. `algorithm_similarity` (s_7): 0.6250 (shared: "resnet 50", "convolutional neural networks")
8. `technology_similarity` (s_8): 0.7120 (shared: "pytorch", "flower", "python", "cuda")

15.4 Step 2: Multiplying Similarities by Ridge Weights

Using the production weights from `models/ridge_fusion_model.joblib`:

1. Title: 0.5842 * +0.13404783 = +0.0783
2. Abstract: 0.6410 * +0.39804354 = +0.2551

3. Problem: 0.4520 * -0.12586323 = -0.0569

4. Objectives: 0.5110 * +0.23673393 = +0.1210

5. Methodology: 0.3890 * +0.00989785 = +0.0039

6. Dataset: 1.0000 * +2.20544639 = +2.2054

7. Algorithm: 0.6250 * -0.06507632 = -0.0407

8. Technology: 0.7120 * -0.14564642 = -0.1037

15.5 Step 3: Additive Linear Reconstruction

Sum of Signed Contributions = +0.0783 + 0.2551 - 0.0569 + 0.1210 + 0.0039 + 2.2054 - 0.0407 - 0.1037

```
= +2.4624
```

Model Intercept (beta_0) = -0.0706

Raw Predicted Similarity = -0.0706 + 2.4624 = 2.3918

Clipped Similarity = clip(2.3918, 0.0, 4.0) = 2.3918

15.6 Step 4: Applying the Novelty Formula

```
Novelty (%) = 100.0 * (1.0 - (s_max / 4.0))
Novelty (%) = 100.0 * (1.0 - (2.3918 / 4.0))
Novelty (%) = 100.0 * (1.0 - 0.59795)
Novelty (%) = 40.205% -> Rounded to 40.2%
```

15.7 Step 5: Assigning the Novelty Assessment Tier

Threshold check:

40.2% falls in the range [25.0%, 50.0%)

Assigned Tier: "Low Novelty (Significant Prior Work)"

15.8 Step 6: XAI Evidence Snippets Generated

In the UI, the student sees:

- Top Match: `SYN-0142` (Similarity 2.392, Novelty 40.2%)
- High Overlap Sections: `dataset` (1.000), `abstract` (0.641), `algorithm` (0.625)
- Differentiating Sections: None under 0.15
- Keyword Snippet Evidence:
- Term: "federated learning"
- Candidate Snippet: "...framework using federated learning across distributed hospital datasets without..."
- Historical Snippet: "...multi-institutional federated learning architecture for clinical diagnostic..."

Chapter 16: Comprehensive Viva & Project Evaluation Q&A;

CHAPTER 16: COMPREHENSIVE VIVA & PROJECT EVALUATION Q&A;

This chapter contains 36 thorough, technically rigorous questions and answers prepared specifically for final-year project evaluations, external examiners, and viva voce defense.

Q1: What is the primary objective of your Academic Project Novelty Detection System?

A1: The objective is to assist academic project review panels and students by evaluating technical proposal novelty against an indexed institutional corpus of prior student work. It breaks proposals into 8 structured sections, computes section-by-section lexical similarities, feeds them to a regularized Ridge regression model, and generates transparent Explainable AI (XAI) evidence.

Q2: What are the eight proposal sections and why are they separated?

A2: The eight sections are Title, Abstract, Problem Statement, Objectives, Proposed Methodology, Dataset, Algorithm/Model, and Technology Tools. They are separated because capstone projects have distinct architectural components. Concatenating all text into one blob conceals where overlap occurs; separating them allows the model to evaluate methodology or dataset overlap independently from generic programming tools.

Q3: What machine learning model is used for similarity scoring?

A3: The system uses an L2-regularized Ridge Regression model (`alpha=1.0`) trained on eight section-level TF-IDF cosine similarities to predict an overall similarity score on a continuous [0.0, 4.0] scale.

Q4: What is TF-IDF and how does it work?

A4: TF-IDF stands for Term Frequency-Inverse Document Frequency. It is a numerical statistic that reflects how important a word is to a specific document within a collection. Term Frequency (TF) measures how often a word occurs in the text, while Inverse Document Frequency (IDF) penalizes words that appear across all projects (e.g., "system", "model") and boosts rare, informative terms (e.g., "plantvillage", "fedavg").

Q5: What are n-grams and why did you choose unigrams and bigrams?

A5: An n-gram is a contiguous sequence of n tokens. A unigram is 1 word ("neural"), while a bigram is 2 consecutive words ("neural network"). Unigrams capture standalone keywords, while bigrams capture technical context and phrases that lose meaning when split.

Q6: Why did you use eight separate TF-IDF vectorizers instead of one shared vectorizer?

A6: Each section has a completely different vocabulary distribution, typical document length, and technical purpose. The `dataset` section contains benchmark proper nouns (83 n-grams), while the `abstract` contains high-level research summaries (454 n-grams). Fitting separate vectorizers ensures document frequencies and IDF weights accurately reflect section-specific importance.

Q7: What is Ridge Regression and why did you choose it over deep neural networks?

A7: Ridge regression is a linear model that minimizes the sum of squared residuals while adding an L2 penalty on coefficient magnitudes ($||w||_2^2$). It was chosen because:

1. Complete Interpretability: Every prediction can be decomposed into additive section contributions ($w_k imes s_k$), satisfying academic transparency.

2. Multicollinearity Stability: The 8 section similarities are correlated. Ridge regression stabilizes weights where standard OLS regression becomes erratic.

3. High Sample Efficiency: Deep neural networks require tens of thousands of real samples and act as opaque black boxes.

Q8: What does the alpha parameter do in Ridge Regression?

A8: Alpha ($■lpha$) is the regularization hyperparameter. It controls the penalty on large coefficients. When $■lpha = 0$, Ridge simplifies to ordinary least squares (OLS). As $■lpha$ increases, coefficients are shrunk closer to zero, reducing model variance and preventing overfitting. Through 5-fold cross-validation, $■lpha=1.0$ and $■lpha=10.0$ were found to provide optimal generalization.

Q9: What is the target variable in your regression model?

A9: The target variable is `expert_similarity_label`, an integer rating from 0 to 4:

- 0: Not Similar
- 1: Slightly Similar
- 2: Moderately Similar
- 3: Highly Similar
- 4: Nearly Duplicate (Identical)

Q10: How many projects and pairs are in your dataset?

A10: The dataset contains 600 unique projects (200 UG Mini, 200 UG Mega, 200 PG Dissertations) across 12 domains. There were originally 3,000 similarity pairs, which were deduplicated to 2,996 clean pairs.

Q11: Are the similarity labels real human faculty annotations or synthetic?

A11: They are 100% synthetic heuristic benchmark labels. The raw dataset explicitly notes: "SYNTHETIC BENCHMARK LABEL — must be replaced/validated by faculty annotation before publication." Across all 13 sheets of `Faculty_Verification_DomainWise.xlsx`, zero faculty evaluations are filled. This is a critical audit disclosure.

Q12: How did you clean the dataset? What duplicate pairs were removed?

A12: In `scripts/clean_dataset.py`, canonical undirected tuples `(min(A, B), max(A, B))` were audited. Exactly 4 redundant rows were removed: 1 exact duplicate (`PAIR-1819`) and 3 reversed duplicate pairs (`PAIR-2023`, `PAIR-2195`, `PAIR-2957`). Both counterparts had 100% matching labels (all label 0).

Q13: What is data leakage and how was it prevented during train/test splitting?

A13: Data leakage occurs when test set information inadvertently influences training. If pairwise rows are randomly split, the same project could appear in both training and test pairs, allowing the model to memorize project vocabulary. We implemented `project_disjoint_split`, which partitions the 600 unique project IDs (480 train, 120 test). Cross-boundary pairs (960 pairs) were discarded.

Q14: How did 5-fold cross-validation work in this project?

A14: The 600 project IDs were split into 5 disjoint folds of 120 projects each (`optimize_models.py`). In each fold, 480 projects formed the training pairs (~1,915 pairs) and 120 projects formed the validation pairs (~117 pairs). TF-IDF vectorizers were fitted strictly on the 480 training projects per fold.

Q15: Why is the holdout RMSE (0.7734) higher than the 5-fold CV mean (0.6511)?

A15: The holdout test set corresponds to Fold 5 of the cross-validation. Individual fold RMSEs ranged from 0.5707 (Fold 3) to 0.7734 (Fold 5). Fold 5 represents the most challenging partition in the data. The 5-fold mean (0.6511) averages across easy and hard folds, whereas the holdout evaluated on Fold 5.

Q16: How are the section weights decided? Did you choose them manually?

A16: The section weights were NOT chosen manually. They were mathematically learned by scikit-learn's `Ridge(alpha=1.0)` to minimize squared prediction errors on the training dataset.

Q17: What are the actual learned weights in your production model?

A17: From `models/ridge_fusion_model.joblib`:

- Intercept: -0.0706

- `dataset`: +2.2054
- `abstract`: +0.3980
- `objectives`: +0.2367
- `title`: +0.1340
- `methodology`: +0.0099
- `algorithm`: -0.0651
- `problem`: -0.1259
- `technology`: -0.1456

Q18: Why does the dataset section have such an enormous weight (+2.2054)?

A18: This is a synthetic dataset artifact. The rule-based generator that created the labels almost always assigned identical datasets to High-Similarity pairs and different datasets to Low-Similarity pairs. The Ridge algorithm discovered that dataset overlap was the strongest statistical predictor of the synthetic label and assigned it over 55% of the total predictive weight.

Q19: Why are some weights negative (e.g., technology = -0.1456)?

A19: This is a suppressor variable effect caused by multicollinearity. Generic tools ("Python", "TensorFlow") and standard algorithms ("CNN") appear ubiquitously across hundreds of completely unrelated projects. If the model had positive weights on tools, unrelated projects would receive false overlap scores. The negative weight suppresses this generic noise.

Q20: What is the mathematical formula for the novelty score?

A20: `Novelty (%) = 100.0 * (1.0 - (max_predicted_similarity / 4.0))`, where `max_predicted_similarity` is clipped to [0.0, 4.0].

Q21: Why is 4.0 used as the denominator in the novelty formula?

A21: Because the regression model was trained on a 0 to 4 ground-truth scale (where 4 represents an identical/duplicate project). Dividing by 4.0 normalizes the score to a [0.0, 1.0] fractional similarity ratio before subtracting from 1.0 to obtain novelty.

Q22: What are the four novelty assessment tiers and their thresholds?

A22:

- 75.0% - 100.0%: "High estimated novelty relative to the current database"
- 50.0% - 74.9%: "Moderate Novelty (Partial Overlap)"
- 25.0% - 49.9%: "Low Novelty (Significant Prior Work)"
- 0.0% - 24.9%: "Very Low Novelty / Potential Duplicate"

Q23: What is the theoretical maximum prediction ceiling of your model?

A23: With an intercept of -0.0706 and coefficient sum of +2.6476, the maximum raw prediction for an exact duplicate is $-0.0706 + 2.6476 = 2.5768$ (out of 4.0). This maps to a minimum novelty of 35.58%. The model can never output a score below 25%, meaning the "Very Low Novelty / Potential Duplicate" tier is mathematically unreachable under current weights!

Q24: Why did an exact copy of a project rank #6 behind near-peers during the audit?

A24: Because of negative weights on `technology_tools` (-0.1456) and `problem_statement` (-0.1259). An exact copy suffers a -0.3366 penalty for sharing 100% of the tools and problem statement, whereas near-peers with differing tools avoid this penalty and score up to 2.6551. This is a non-monotonicity limitation.

Q25: What is candidate self-exclusion and how does it work?

A25: When evaluating a proposal, the system must not match a project against itself. `NoveltyExplainer` checks:

1. Case-insensitive project ID match.

2. Case-insensitive exact title match.

3. Identical feature vector match (all 8 similarities >= 0.9999).

Matching records are masked out, ensuring the candidate is only evaluated against prior peer projects.

Q26: What is Explainable AI (XAI) in your system and how is it implemented?

A26: XAI provides human-understandable evidence explaining why a prediction was made. Our system implements four layers:

1. Ranked Sections Table (ordering sections by overlap degree).

2. Additive Model Attributions (displaying exact $w_k imes s_k$ contributions).

3. Shared Informative Terms (displaying common technical n-grams filtered for stop words).

4. Source-Verified Snippets (displaying verbatim text windows centered around shared keywords with clean word boundaries).

Q27: How do you extract and verify keyword snippets?

A27: `extract_snippet` extracts a 45-character window around the keyword and snaps outward to the nearest whitespace to prevent mid-word truncations. `verify_snippet` strictly confirms that the snippet body is an exact substring of the original source record.

Q28: How does the system handle user proposal submissions without contaminating the database?

A28: Scoring a proposal via `/api/score` is strictly read-only and in-memory. If a user explicitly clicks "Save Proposal", `/api/save` writes the proposal and its report to a separate SQLite database (`data/proposals.db`). The 600 baseline reference projects in Joblib are never mutated.

Q29: How does the system handle duplicate submissions in the database?

A29: In `src/storage/db.py`, before inserting a new proposal, the database checks for an existing record with the same normalized alphanumeric title. If found, it raises `DuplicateProposalError` and FastAPI returns HTTP 409 Conflict.

Q30: Why does your model fail when a proposal is paraphrased?

A30: Because TF-IDF is a lexical bag-of-words model that measures exact word matching. When concepts are paraphrased using different synonyms, TF-IDF n-gram overlap drops to near zero, causing the model to output a false 99.6% novelty score.

Q31: What is the single most important limitation of your system?

A31: 100% of the ground-truth similarity labels are synthetic heuristic rules, and zero real faculty evaluations exist in the dataset. High evaluation metrics simply reflect the model learning the synthetic labeling rule.

Q32: How would you improve and scientifically validate this project in the future?

A32:

1. Collect real faculty annotations for 300 project pairs in `Faculty_Verification_DomainWise.xlsx`.

2. Retrain using Non-Negative Least Squares (NNLS) to constrain $w_i \ge 0$, fixing non-monotonicity.

3. Calibrate the output scale so identical projects map to 4.0 (0% novelty).

4. Integrate dense Sentence-Transformers (`all-MiniLM-L6-v2` or `Specter2`) to detect paraphrasing.

5. Evaluate on a true, sealed holdout of real student projects.

Q33: Can your system be used to prove plagiarism or accuse a student of copying?

A33: Absolutely not. The system estimates lexical similarity relative to a local reference database. It cannot establish intent, uncredited copying, or academic dishonesty. Shared terminology often occurs naturally because students use standard algorithms or datasets.

Q34: How fast does the model evaluate a proposal at runtime?

A34: Because TF-IDF vectorization and Ridge regression are simple matrix operations, transforming 8 sections and computing cosine similarity against 600 projects takes less than 35 milliseconds.

Q35: What libraries are used in the backend?

A35: Python 3.12, FastAPI, Pydantic v2, Uvicorn, Scikit-Learn (TF-IDF & Ridge), Pandas, NumPy, SciPy, Joblib, and SQLite3.

Q36: What libraries are used in the frontend?

A36: React 19, Vite 6, Tailwind CSS v4, Recharts (for similarity and Ridge contribution bar charts), and Lucide React (for icons).

Chapter 17: Glossary, Summary Cheat Sheet & Viva Advice

CHAPTER 17: GLOSSARY, SUMMARY CHEAT SHEET & VIVA ADVICE

17.1 Technical Glossary (25 Key Terms)

1. Token: An individual word or character sequence extracted during text processing.

2. N-Gram: A contiguous sequence of n tokens. Unigram = 1 word; Bigram = 2 words.

3. TF (Term Frequency): The frequency of a word within a specific section text.

4. IDF (Inverse Document Frequency): A logarithmic metric penalizing common words across the corpus.

5. TF-IDF: The product of TF and IDF, representing word importance in sparse vector format.

6. L2 Normalization: Scaling a vector so its Euclidean length equals 1.0.

7. Cosine Similarity: The cosine of the angle between two normalized vectors (dot product), ranging [0, 1].

8. Ridge Regression: Linear regression augmented with an L2 penalty on coefficient magnitudes.

9. Regularization (L2): A penalty term (alpha * sum(w^2)) added to the loss function to prevent overfitting.

10. Alpha: The hyperparameter controlling the strength of L2 regularization in Ridge regression.

11. Intercept (beta_0): The baseline predicted value when all input features are zero (-0.0706).

12. Coefficient (w_i): The learned multiplier for each section similarity in the linear model.

13. Suppressor Variable: A feature with negative weight that cancels out irrelevant noise from collinear features.

14. Multicollinearity: High linear correlation among independent feature variables.

15. Project-Disjoint Split: Partitioning data by unique project IDs to guarantee zero train/test leakage.

16. Cross-Boundary Pairs: Pairs where one project is in Train and the other is in Test (must be discarded).

17. MAE (Mean Absolute Error): Average absolute difference between predicted and true labels.

18. RMSE (Root Mean Squared Error): Square root of mean squared error, penalizing large outliers.

19. R2 Score: Proportion of variance in target labels explained by the model.

20. Spearman Rho: Non-parametric rank correlation measuring monotonic ordering.

21. Novelty Percentage: 100 * (1 - predicted_similarity / 4.0).

22. Explainable AI (XAI): Techniques that make machine learning decision logic transparent and inspectable.

23. Additive Reconstruction: Verifying that Intercept + sum(Contributions) == Raw Prediction.

24. Candidate Self-Exclusion: Filtering out the query project itself to prevent trivial 100% self-similarity.

25. Synthetic Ground Truth: Labels generated by heuristic formulas rather than human expert adjudications.

17.2 The Entire ML Pipeline on One Page

1. Input: 8 structured text sections of an academic proposal.

2. Vectorization: 8 pre-fitted TfidfVectorizers (ngram_range=(1,2), stop_words='english') transform texts into 8 sparse vectors.

3. Comparison: Dot product against 600 baseline project vectors yields 8 cosine similarity scores per historical project.

4. Linear Fusion: Ridge regression combines similarities:

```
y_raw = -0.0706 + (0.134*s_title) + (0.398*s_abstract) - (0.126*s_prob) + (0.237*s_obj)
+ (0.010*s_meth) + (2.205*s_data) - (0.065*s_algo) - (0.146*s_tech)
```

5. Filtering: Candidate project is self-excluded by ID, title, or vector identity.

6. Ranking: Historical projects are sorted by predicted similarity descending; top 5 are selected.

7. Novelty Formulation: s_max is the highest similarity among remaining projects.

```
Novelty (%) = 100 * (1.0 - (s_max / 4.0))
```

8. XAI Generation: Sections are ranked; shared n-grams and source-verified text snippets are extracted.

9. Database Segregation: Scored proposal is displayed in React; stored in SQLite ONLY if user clicks "Save".

17.3 Most Important Formulas to Memorize

1. Novelty Formula:

```
Novelty (%) = 100.0 * ( 1.0 - (s_max / 4.0) )
```

2. Ridge Prediction Formula:

```
y_raw = beta_0 + sum_{i=1}^8 ( w_i * s_i )
```

3. Additive Contribution Formula:

```
contribution_i = w_i * s_i
y_raw == beta_0 + sum_{i=1}^8 contribution_i
```

4. Cosine Similarity Formula:

```
s_i = ( v_cand . v_hist ) / ( ||v_cand|| * ||v_hist|| )
```

17.4 Most Important Numbers to Memorize

- 600: Total unique historical projects in the reference database (200 Mini, 200 Mega, 200 PG).
- 2,996: Total clean pairwise comparisons used in training (4 duplicate pairs removed from 3,000).
- 480 / 120: The 80/20 project-disjoint split (480 training projects = 1,916 pairs; 120 test projects = 120 pairs).
- 1,736: Total learned vocabulary size across all eight TF-IDF vectorizers.
- -0.0706: The model intercept ($■eta_0$).
- +2.2054: The dataset coefficient ($w_{dataset}$), representing over 55% of total predictive weight.
- +2.6476: The sum of all eight learned coefficients.
- 2.5768: The maximum possible raw prediction for a 100% identical duplicate project.
- 35.58%: The minimum possible novelty score for an exact duplicate (meaning < 25% tier is unreachable).
- 0.6511: 5-Fold Cross-Validation Mean RMSE (± 0.0910).
- 0.7734: Final Holdout RMSE (matches Fold 5 of cross-validation).
- 0: Total number of real faculty annotations in the raw verification workbook.

17.5 Final Viva Advice

- Be Proud of What You Built: You built a working, end-to-end, full-stack application with React, Vite, Tailwind, FastAPI, SQLite, and an Explainable AI pipeline with sub-35ms response times.

- Be Scientifically Honest: External examiners love students who understand the limitations of their own models. If asked, "Is this ready for real-world deployment?", answer:

"No, sir/ma'am. The software architecture is complete and mathematically verified, but the model was trained on synthetic benchmark labels. Before real-world deployment, faculty members must annotate real project pairs in our verification package, and we should integrate dense semantic embeddings to defend against paraphrasing."

Answering with scientific honesty demonstrates mature engineering and academic integrity!